

Simulating non-trivial incompressible flows with a quantum lattice Boltzmann algorithm

David Jennings*, Kamil Korzekwa*[†], Matteo Lostaglio*, and Paul Mannix*[†]
PsiQuantum, 700 Hansen Way, Palo Alto, CA 94304, USA

Richard Ashworth[‡], Emanuele Marsili[§], and Stephen Rolston[¶]
Airbus Operations Ltd, Pegasus House Aerospace Avenue, Filton, Bristol, UK

Quantum algorithms have been identified as a potential means to accelerate computational fluid dynamics (CFD) simulations, with the lattice Boltzmann method (LBM) being a promising candidate for realizing quantum speedups. Here, we extend the recent quantum algorithm for the incompressible LBM to account for realistic fluid dynamics setups by incorporating walls, inlets, outlets, and external forcing. We analyze the associated complexity cost and show that these modifications preserve the asymptotic scaling, and potential quantum advantage, of the original algorithm. Moreover, to support our theoretical analysis, we provide a classical numerical study illustrating the accuracy, complexity, and convergence of the algorithm for representative incompressible-flow cases, including the driven Taylor–Green vortex, the lid-driven cavity flow, and the flow past a cylinder. Our results provide a pathway to accurate quantum simulation of nonlinear fluid dynamics, and a framework for extending quantum LBM to more challenging flow configurations.

I. Introduction

In settings where the understanding and prediction of the behavior of fluid flow is the ultimate aim and, in particular, when experiments are either impossible (e.g., in oceanography and metrology) or prohibitively expensive/time-consuming (e.g., building and testing multiple prototypes in wind-tunnels), computational fluid dynamics (CFD) models have become indispensable [1]. Due to the challenges associated with numerically modeling the non-trivial flow around a complex geometry with multiple different boundary conditions, a number of specialized high performance CFD codes have been developed. Albeit based on fundamentally different numerical methods including the finite-volume [2], spectral-element [3] and lattice Boltzmann method [4], these codes are highly optimized for parallelized execution on CPU and GPU architectures. As a result, it has been possible to simulate applications as diverse as landing-gear, wind farms and turbo-machinery, respectively. However, as these simulations seek to grow larger and better resolve the flow physics, computational constraints emerge. Notwithstanding the computational complexity increasing at least cubically with the Reynolds number Re , representing the flow field for a large computation demands also distributed memory. This, in turn, requires computational operations to be kept as local as possible in order to minimize the communication overhead [5]. On this basis, tensor network approximations based on matrix product states [6], super-resolution machine learning methods [7] as well as quantum CFD algorithms [8] have emerged.

Quantum computing offers a fundamentally different way to process information that is expected to provide transformative (in some cases exponential) improvements over what is possible with ‘classical’ computing, i.e., computational devices (such as today’s supercomputers) that process information via classical physics [9–13]. Instead of bits that are 0 or 1, a quantum computer uses quantum states that can explore many possibilities at once, and carefully designed algorithms ‘interfere’ these possibilities to amplify the desired solution. Over the past decade, researchers have built algorithms that target tasks central to scientific computing, such as solving large linear systems [14–16], estimating observables [17–19], and evolving dynamical models [20–26]. For nonlinear dynamics, ubiquitous in fluid flow, the nonlinearities must be put into a form amenable to a quantum computer. To achieve this, one can lift the nonlinear problem into a higher-dimensional *linear* form (e.g., via embedding/expansion methods [27]) and then solve

*Quantum Applications Team, PsiQuantum, 700 Hansen Way, Palo Alto, CA 94304, USA

[†]Lead authors’ emails: kkorzekwa@psiquantum.com and pmannix@psiquantum.com (authors are listed alphabetically within each affiliation).

[‡]Research Scientist, Central Research and Technology, Airbus Operations Ltd, Pegasus House Aerospace Avenue, Filton, Bristol, UK

[§]Research Scientist, Central Research and Technology, Airbus Operations Ltd, Pegasus House Aerospace Avenue, Filton, Bristol, UK

[¶]Research Project Leader, Central Research and Technology, Airbus Operations Ltd, Pegasus House Aerospace Avenue, Filton, Bristol, UK

the resulting system with quantum linear-algebra routines, which are known to circumvent the curse of dimensionality. Since the quantum-encoded data is a potentially exponentially large vector, the aim is not to print out the entire flow field, but to extract specific features or observables faster than classical methods might allow, especially for very large problem sizes. That said, potential speedups come with caveats: data must be encoded efficiently, the lifted system must be well-conditioned enough for the solver to work, and today's hardware remains noisy, so most proposals target future, fault-tolerant machines [28–33]. Nevertheless, emerging quantum algorithms are a promising tool-in-the-making for selected, structured nonlinear problems, which are expected to supplement existing CFD methods in novel and impactful ways.

One prominent example of a structured nonlinear problem is given by the *lattice Boltzmann equation* (LBE), whose nonlinear strength scales with the Mach number Ma , and whose memory demands are more intensive as compared with those of other CFD methods. On this basis, a number of quantum algorithms employing Carleman embedding/expansion methods have been proposed in recent years [34–37]. In Ref. [38], it was shown that these prior works did not properly address the crucial convergence of the Carleman embedding, which is essential for resolving nonlinear physics properly. The work also highlighted that prior algorithms suffered from inefficient information-extraction from the quantum state, as well as other algorithmic bottlenecks, such as unreasonably small time-step requirements. In order to circumvent these obstacles, the authors of Ref. [38] developed a quantum algorithm for the *incompressible* lattice Boltzmann equation [39, 40] and provided evidence that a modest quantum advantage can occur over classical methods, albeit with caveats that require further study and development.

While Ref. [38] concentrated on building the theoretical framework for the quantum lattice Boltzmann algorithm, connecting these results to real-world problems of industrial relevance requires incorporating non-trivial boundary conditions and external driving. In this paper, we present the modifications required to extend Ref. [38] so that walls, inlets, outlets, and forcing can be accommodated. We explain the corresponding changes to the quantum algorithm, assess their complexity cost, and argue that the introduced extensions should not significantly slow down the designed quantum algorithm. In addition, we conduct a thorough numerical assessment of the Carleman convergence to the correct nonlinear dynamics. This is performed by computing solutions of the driven Taylor-Green vortex, the lid-driven cavity flow, and the flow past a cylinder.

The structure of this work is as follows. First, in Sec. II, we briefly review the incompressible LBM and its associated quantum algorithm developed in Ref. [38], introducing the necessary notation. Then, in Sec. III, we explain how, and at what complexity cost, one can extend this algorithm to include non-trivial boundary conditions in the form of walls, inlets and outlets, as well as how to add driving. In Sec. IV, we present the convergence and complexity analysis for exemplary CFD settings. Finally, Sec. V contains conclusions and outlook for future work, while the Appendix contains extra technical details to make this work self-contained.

II. Existing framework

A. Lattice Boltzmann method

The *lattice Boltzmann equation* (LBE) is a kinetic model of fluid dynamics that describes the fluid mesoscopically [41]. It conserves mass and momentum by construction and its low-order moments recover the Navier-Stokes equations in the regime where the characteristic length scales of the problem are much larger than the molecular mean path. Within LBE, the fluid is modeled at time t and position $\mathbf{r}$ in D -dimensional space by a vector-valued function $\mathbf{g}(\mathbf{r}, t)$, whose components $g_m(\mathbf{r}, t)$, with $m \in \{1, \dots, Q\}$, are proportional to the probability density of finding a fluid particle with a discrete velocity $\mathbf{e}_m$. LBE is derived from the *discrete Boltzmann equation* (DBE) describing continuous dynamics via:

$$\partial_t g_m(\mathbf{r}, t) + \mathbf{e}_m \cdot \nabla g_m(\mathbf{r}, t) = \Omega_m(\mathbf{r}, t), \quad (1)$$

where $\mathbf{e}_m \cdot \nabla g_m$ is an advection term that describes *streaming*, i.e., the flow of the fluid in space, while the collision operator Ω_m describes *collisions* that are local in space. We adopt the Bhatnagar-Gross-Krook (BGK) operator [42]:

$$\Omega_m(\mathbf{r}, t) = -\frac{1}{\tau} [g_m(\mathbf{r}, t) - g_m^{\text{eq}}(\mathbf{r}, t)], \quad (2)$$

where τ is the relaxation time determining the speed of equilibration, and $\mathbf{g}^{\text{eq}}$ is the Taylor expansion of the local Maxwell equilibrium up to second order terms, which for an incompressible flow is given by [39]:

$$g_m^{\text{eq}}(\mathbf{r}, t) = \frac{p}{c_s^2} w_m + \rho_0 \left(\frac{\mathbf{e}_m \cdot \mathbf{u}(\mathbf{r}, t)}{c_s^2} + \frac{(\mathbf{e}_m \cdot \mathbf{u}(\mathbf{r}, t))^2}{2c_s^4} - \frac{|\mathbf{u}(\mathbf{r}, t)|^2}{2c_s^2} \right) w_m + O(Ma^3). \quad (3)$$

In the above, ρ_0 denotes the constant fluid density, c_s the speed of sound, $\text{Ma} := |\mathbf{u}|/c_s$ the Mach number, and $\mathbf{w}$ is a vector of Maxwell weightings for the different discrete velocities satisfying $w_m \geq 0$ and $\sum_m w_m = 1$. The pressure p and velocity vector $\mathbf{u}$ of the fluid are recovered as moments via:

$$p(\mathbf{r}, t) := c_s^2 \sum_{m=1}^Q g_m(\mathbf{r}, t), \quad \mathbf{u}(\mathbf{r}, t) := \frac{1}{\rho_0} \sum_{m=1}^Q g_m(\mathbf{r}, t) \mathbf{e}_m. \quad (4)$$

As shown in Ref. [39], this formulation recovers the Navier-Stokes equations, albeit with the divergence-free condition approximated as $\nabla \cdot \mathbf{u} = \frac{1}{c_s^2} \partial_t p \sim O(\text{Ma}^2)$.^{*} Therefore, at the expense of discarding weak compressibility effects, which for $\text{Ma} \ll 1$ are already negligible, an $O(\text{Ma}^2)$ incompressible LBE formulation whose collision integral is quadratic in g_m is obtained.

To obtain the numerical solution of Eqs. (1) - (4), the system must be discretized in space, $r_i \in [\Delta x, 2\Delta x, \dots, N_i \Delta x]$, and in time, $t \in [0, \Delta t, 2\Delta t, \dots, N_t \Delta t]$. Integrating the left hand side of Eq. (1) using the method of characteristics and approximating the integral on the right side of Eq. (1) using the trapezoidal rule, one obtains:

$$g_m(\mathbf{r} + \mathbf{e}_m \Delta t, t + \Delta t) = g_m(\mathbf{r}, t) - \frac{\Delta t}{\tau + \Delta t/2} [g_m(\mathbf{r}, t) - g_m^{\text{eq}}(\mathbf{r}, t)], \quad (5)$$

which is second order accurate in both space and time [41]. To determine how Eq. (5) relates to the macroscopic Navier-Stokes equations, Chapman-Enskog analysis must be applied, which shows that the kinematic shear viscosity,

$$\nu = c_s^2 \left(\tau - \frac{\Delta t}{2} \right), \quad (6)$$

is a function of the relaxation time, time-step and speed of sound. As discussed in Ref. [41], the factor of $(\tau - \Delta t/2)$ appearing in Eqs. (5) and (6) is not general, but rather a consequence of the time-discretization employed. Because we have used a second-order time discretization, the relaxation time τ is shifted by $\Delta t/2$. Although physically significant length- and time-scales are typically chosen, the system from Eqs. (3) - (6) is instead made non-dimensional by choosing the lattice spacing Δx and time step Δt (also known as lattice units) along with the reference density ρ_0 , such that:

$$t = t^* \Delta t, \quad T = N_t \Delta t, \quad \tau = \tau^* \Delta t, \quad \mathbf{r} = \mathbf{r}^* \Delta x, \quad L_i = N_i \Delta x, \quad \mathbf{e}_m = \mathbf{e}_m^* \frac{\Delta x}{\Delta t}, \quad \mathbf{u} = \mathbf{u}^* \frac{\Delta x}{\Delta t}, \quad p = p^* c_s^2 \rho_0, \quad (7)$$

where superscript star denotes a non-dimensional variable, L_i is the length of the i^{th} side of the domain and T is the size of time-interval considered. This choice restricts $t^* \in \{0, \dots, N_t\}$ and $r_i^* \in \{1, \dots, N_i\}$ to integer values, such that for an incompressible isothermal flow $c_s = \frac{1}{\sqrt{3}} \frac{\Delta x}{\Delta t}$ and $\mathbf{e}_m^*$, w_m are given by Table 2 in the Appendix.

Anticipating the quantum formulation of Eq. (5), the system state at each time step t^* can be represented as:

$$\mathbf{g}(t^*) = \sum_{\mathbf{r}^*} \sum_{m=1}^Q g_m(\mathbf{r}^*, t^*) |\mathbf{r}^*\rangle \otimes |m\rangle, \quad (8)$$

where $\otimes$ denotes the tensor product and $|\mathbf{r}^*\rangle, |m\rangle$ denote position and velocity registers, respectively, using the Dirac ket notation. The dimension of $\mathbf{g}$ is $d = NQ$, where N is the total number of spatial lattice points. Equation (5) can be then be divided into collision (C) and streaming (S) steps, and rewritten in a matrix form as:

$$\mathbf{g}^C(t^*) = (I + F_1) \mathbf{g}(t^*) + F_2 \mathbf{g}(t^*)^{\otimes 2}, \quad (9a)$$

$$\mathbf{g}(t^* + 1) = S \mathbf{g}^C(t^*), \quad (9b)$$

where F_k are $d \times d^k$ matrices describing linear and quadratic contributions from collisions and S is a $d \times d$ permutation matrix encoding streaming, see the Appendix for details.

^{*}Although this is suitable for steady or slowly varying flows, the collision operator proposed in Ref. [40] (also quadratic in g_m) is by contrast exact and therefore preferable for time-dependent flows.

B. Carleman embedding of the nonlinear dynamics

The lattice Boltzmann equation is nonlinear, whereas quantum algorithms running on quantum computers are built from linear (unitary) operations, so there is a structural mismatch. The way in which this mismatch is resolved is by embedding the nonlinear dynamics into a much higher-dimensional linear system, whose variables are functions of the original ones. While this lifted system can be overwhelmingly large for classical solvers, quantum circuits act on superpositions and apply linear operators implicitly, so the cost need not scale with the full dimension in the same way as explicit classical representations. In practice, this makes it viable to work with high-dimensional embeddings on a quantum computer, provided the embedding can be implemented efficiently and remains well-behaved.

In order for the incompressible LBE problem, captured by Eqs. (9a)-(9b), to be amenable for the quantum computer, we use the Carleman embedding technique [27]. Instead of representing the state of the fluid at each moment in time by a d -dimensional vector $\mathbf{g}$, it is now represented by the *Carleman vector*:

$$\mathbf{y}(t^\star) := [\mathbf{y}_1(t^\star), \mathbf{y}_2(t^\star), \dots, \mathbf{y}_{N_C}(t^\star)], \quad \mathbf{y}_k(0) = \mathbf{g}^{\otimes k}(0) \quad (10)$$

of dimension

$$d_C := d + d^2 + \dots + d^{N_C} = \frac{d(d^{N_C} - 1)}{d - 1}, \quad (11)$$

which carries information about all powers of the incompressible LBE evolution up to the N_C -th power. Here, N_C is the finite Carleman truncation order leading to Carleman truncation error ϵ_C , i.e., the evolution obtained from a truncated linear system will generally differ from the original LBE dynamics.

The update rules for a d -dimensional vector $\mathbf{g}(t^\star)$ from Eqs. (9a)-(9b), can now be approximated by a linear update rule for a d_C -dimensional vector $\mathbf{y}(t^\star)$ [38]:

$$\mathbf{y}(t^\star + 1) = \mathcal{S}\mathcal{C}\mathbf{y}(t^\star), \quad (12)$$

where $\mathcal{S}$ and $\mathcal{C}$ are $d_C \times d_C$ Carleman streaming and collision matrices, see the Appendix for details. The above recurrence relation has a clear solution given by:

$$\mathbf{y}(t^\star) = (\mathcal{S}\mathcal{C})^{t^\star} \mathbf{y}_{\text{ini}}, \quad \mathbf{y}_{\text{ini}} = [\mathbf{g}(0), \mathbf{g}(0)^{\otimes 2}, \dots, \mathbf{g}(0)^{\otimes N_C}]. \quad (13)$$

Therefore, for the initial state $\mathbf{g}(0)$, the state of the system after N_t streaming and collision steps is approximated by:

$$\mathbf{g}(N_t) \approx_{\epsilon_C} [(\mathcal{S}\mathcal{C})^{N_t} \mathbf{y}_{\text{ini}}]_1, \quad (14)$$

where subscript 1 indicates the first block of the Carleman vector.

Next, the LBE problem with N_t streaming and collision steps can be written in the form of the following linear system of dimension $d_C(N_t + 1)$:

$$\begin{pmatrix} I & 0 & 0 & \dots & 0 & 0 \\ -\mathcal{S}\mathcal{C} & I & 0 & \dots & 0 & 0 \\ 0 & -\mathcal{S}\mathcal{C} & I & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & I & 0 \\ 0 & 0 & 0 & \dots & -\mathcal{S}\mathcal{C} & I \end{pmatrix} \begin{pmatrix} \mathbf{y}(0) \\ \mathbf{y}(1) \\ \mathbf{y}(2) \\ \vdots \\ \mathbf{y}(N_t - 1) \\ \mathbf{y}(N_t) \end{pmatrix} = \begin{pmatrix} \mathbf{y}_{\text{ini}} \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{pmatrix}, \quad (15)$$

or in a more compact form as

$$\mathbf{A}\mathbf{Y} = \mathbf{b}. \quad (16)$$

The vector $\mathbf{Y}$ is the *Carleman history state*, carrying information about Carleman vector at all times during the simulated evolution:

$$\mathbf{Y} = \mathbf{A}^{-1}\mathbf{b} = (\mathbf{y}_{\text{ini}}, \mathcal{S}\mathcal{C}\mathbf{y}_{\text{ini}}, (\mathcal{S}\mathcal{C})^2\mathbf{y}_{\text{ini}}, \dots, (\mathcal{S}\mathcal{C})^{N_t-1}\mathbf{y}_{\text{ini}}, (\mathcal{S}\mathcal{C})^{N_t}\mathbf{y}_{\text{ini}})^\top. \quad (17)$$

The problem therefore reduces to performing a matrix inversion, and outputting the solution to the above set of linear equations. Although the solution vector $\mathbf{Y}$ can be exponentially large, as a function of the Carleman truncation scale, one can now exploit the particular features of a quantum computer that allow one to process such vectors efficiently. More precisely, it is known that *quantum linear solvers* (QLS) have exponential speedups over classical methods for certain structured problems [14] and, more recently, it has been shown that they can also provide exponential speedups in the context of simulating nonlinear dynamical systems via the above Carleman method [33].

C. Solving exponentially large linear systems on a quantum computer

Quantum algorithms for solving systems of linear equations as in Eq. (16) typically return the solution in an encoded (quantum) form rather than a full list of numbers [14–16, 43], so they are most useful when one only needs certain features, like averages or correlations, rather than every entry. Their performance also depends on how well-behaved the equations are (in terms of the condition number $\kappa_A = \|A\| \cdot \|A^{-1}\|$) and on whether the problem has structure that allows one to load data and apply matrices efficiently. It is rigorously established that promising quantum speedups exist, but only for the right kinds of large, structured problems [14].

The ability of quantum linear solvers to provide large speedups relies crucially on the output of a quantum computer, which differs radically from what one has from a classical computer. This is because the solution $\mathbf{Y} = A^{-1}\mathbf{b}$ is encoded in a *quantum state*. Physically, this corresponds to a quantum mechanical system (such as a system of photons, superconductors, atoms, etc.) being in a superposition of two or more classical states, e.g., a particle being in a superposition of two locations A and B . Mathematically, we write this superposition state in the Dirac notation as a vector $|\psi\rangle = a|A\rangle + b|B\rangle$, where a and b are complex numbers that obey $|a|^2 + |b|^2 = 1$. Within quantum theory, the classical states are represented as orthonormal basis vectors, and so in this basis we have that $|\psi\rangle$ is the vector with components (a, b) . More generally, we label classical states with integers, and a general d -dimensional quantum state takes the form:

$$|\psi\rangle = a_0|0\rangle + a_1|1\rangle + \dots + a_{d-1}|d-1\rangle, \quad (18)$$

with the condition $\sum_j |a_j|^2 = 1$. If we measure the system, we collapse the quantum state probabilistically onto one of the states $|k\rangle$, and the probability that this occurs is $p_k = |a_k|^2$. Therefore, the normalization condition is simply the condition that measurement probabilities in quantum theory sum to 1. Upon getting outcome k , the quantum state collapses $|\psi\rangle \rightarrow |k\rangle$. At the level of linear algebra, this can be described (up to normalizing the vector) by the application of a projection matrix:

$$(a_0, \dots, a_{d-1}) \rightarrow P_k(a_0, \dots, a_{d-1}) = (0, 0, \dots, 0, a_k, 0, \dots, 0). \quad (19)$$

Here, P_k is the projection matrix onto the k -th basis vector that can be written in the Dirac notation as $P_k = |k\rangle\langle k|$.

More generally, we have the compact Dirac notation convention defining rank-1 matrices $|m\rangle\langle k|$ given by:

$$|m\rangle\langle k| \sum_j a_j |j\rangle = \sum_j a_j |m\rangle\langle k| j\rangle = a_k |m\rangle, \quad (20)$$

in other words $\langle k|j\rangle = \delta_{j,k}$ for any k, j . Given two quantum states $|\psi\rangle = \sum_j a_j |j\rangle$ and $|\phi\rangle = \sum_j b_j |j\rangle$, we then define the conjugate transposed vector $\langle\phi| := \sum_j b_j^* \langle j|$, the matrix

$$|\psi\rangle\langle\phi| = \sum_{j,k} a_j b_k^* |j\rangle\langle k|, \quad (21)$$

and the complex number obtained from taking the inner product of these two vectors:

$$\langle\phi|\psi\rangle = \sum_{j,k} a_j b_k^* \delta_{j,k} = \sum_k a_k b_k^*. \quad (22)$$

We also can have multiple copies of quantum states and denote, e.g., two systems with the first in the state $|\psi\rangle$ and the second in the state $|\phi\rangle$ as $|\psi\rangle \otimes |\phi\rangle$. In other words: multiple copies are denoted with terms separated by tensor products, and $A^{\otimes n} = A \otimes A \otimes \dots \otimes A$ denotes n copies of A . In this setting, the rules of quantum theory correspond to linear transformations (matrices on vectors) and linear algebra manipulations (compositions of matrices, inner products, tensor products, etc.).

To implement quantum algorithms on a quantum computer, every component must be encoded in a *quantum circuit* (a sequence of instructions as to how to manipulate the quantum system). Mathematically, a quantum circuit is described by an exponentially large unitary matrix that acts on quantum states. This exponential matrix could describe various data, such as vectors and matrices. For example, the primitive of *state preparation* involves a quantum circuit $U_{\mathbf{b}}$ that acts on a fiducial vector $|0\rangle = (1, 0, 0, \dots, 0)$ and prepares the new vector $|\mathbf{b}\rangle = U_{\mathbf{b}}|0\rangle = \frac{1}{\|\mathbf{b}\|}(b_0, b_1, \dots, b_{d-1})$. In other words, the first column of $U_{\mathbf{b}}$ is simply the normalized state vector $|\mathbf{b}\rangle$. Another key ingredient is the encoding of an arbitrary matrix A inside a unitary circuit. In order to respect unitarity, the matrix A is typically rescaled $A \rightarrow A/\alpha_A$ by some positive constant $\alpha_A \geq \|A\|$, and then a quantum circuit U_A is constructed such that the top-left block of the matrix is identical to A/α_A ; this is called a *unitary block-encoding* of the matrix A .

A standard approach to solving a linear system $AY = \mathbf{b}$ on a quantum computer [14, 15, 43] assumes access to (i) an efficient *unitary state preparation* $U_{\mathbf{b}}$ such that $U_{\mathbf{b}} |0\rangle = |\mathbf{b}\rangle$, and (ii) a *block-encoding* U_A of A , i.e., a unitary U_A acting on $\log d + n_A$ qubits whose top-left block equals A/α_A for some *block-encoding prefactor* $\alpha_A \geq \|A\|$. Given $(U_{\mathbf{b}}, U_A)$, modern *quantum linear solvers* output a state proportional to $A^{-1} |\mathbf{b}\rangle$ as a normalized quantum state. Optimal quantum linear solvers prepare the solution $|Y\rangle$ within error ϵ using $O(\alpha_A \kappa_A \log \frac{1}{\epsilon})$ applications of $U_A, U_A^\dagger$ and of the preparation unitaries $U_{\mathbf{b}}, U_{\mathbf{b}}^\dagger$, where κ_A is the condition number of A on the relevant subspace. The dependence on both the condition number and the block-encoding prefactor is linear; thus large α_A directly inflates runtime. In practice, (quantum or classical) preconditioning can reduce κ_A , but any gains must be balanced against changes in α_A and additional oracle costs.

One final subtlety exists for QLS: the output. The output of the quantum linear solver applied to the Carleman system will be a *quantum state* $|Y\rangle$ of the quantum computer. This is quite distinct from the output of a classical computer, which can, e.g., print the data on a screen to be perused. Instead, the quantum output $|Y\rangle$ is an exponentially large vector that is ‘stored’ in the quantum computer (e.g., for a quantum computer with 1000 qubits, the vector has dimension $2^{1000} \approx 10^{300}$). In order to extract meaningful information, one must perform a carefully chosen measurement that extracts a much smaller amount of information from $|Y\rangle$. The full end-to-end quantum simulation must therefore build in the exact information that is required into the full pipeline, and the information extraction becomes itself a potential source of inefficiency. While we do not discuss that in this work, we refer the interested reader to Ref. [38], where it is shown how, and at what complexity cost, one can extract the drag force on an obstacle from a quantum state $|Y\rangle$.

III. Extensions to nontrivial flows and geometries

Having introduced the incompressible Carleman LBM algorithm from Ref. [38] and discussed how to recover its solution using QLS, we now extend it to accommodate non-trivial geometries and driving via a body forcing or moving boundaries. These extensions will allow us to investigate the performance of our quantum algorithm on benchmark CFD problems, such as the forced Taylor-Green vortex, the lid-driven cavity flow and the flow past a cylinder, considered in Sec. IV.

A. Geometry

1. Extending the model

For periodic boundary conditions, there are only fluid nodes and the system can be encoded in the streaming matrix:

$$S = \sum_{\mathbf{r}^\star} \sum_{m=1}^Q |\mathbf{r}^\star + \mathbf{e}_m^\star\rangle\langle\mathbf{r}^\star| \otimes |m\rangle\langle m|, \quad (23)$$

which translates populations g_m from position $\mathbf{r}^\star$ and with a discrete velocity m to the next spatial position $\mathbf{r}^\star + \mathbf{e}_m^\star$, where the addition should be interpreted as modulo the number of lattice points per direction, thus encoding the periodic domain. We want to modify this streaming matrix in order to account for the no-slip (Dirichlet) and free-slip (Neumann) boundary conditions required to consider domains with walls, inlets and outlets.

The effect of a Dirichlet boundary condition can be modeled via the bounce-back rule following Ref. [41], whereby one prescribes these macroscopic boundary conditions via the populations as:

$$g_{\bar{m}}(\mathbf{r}_b^\star, t^\star) = g_m^C(\mathbf{r}_b^\star, t) - 2w_m \frac{\mathbf{e}_m^\star \cdot \mathbf{u}_w^\star}{c_s^{\star 2}}. \quad (24)$$

Here, $\mathbf{r}_b^\star$ is the boundary node, g_m^C the post-collision population, $\bar{m}$ the reflection of/population opposite m , and $\mathbf{u}_w^\star$ the velocity vector of the wall located at $\mathbf{r}_w^\star = \mathbf{r}_b^\star + \frac{1}{2}\mathbf{e}_m^\star$. As the second term in Eq. (24) accounts for the motion imparted by a moving wall, we will include it in the driving matrix described in section III.B. The effect of a Neumann boundary condition can be modeled via the non-equilibrium extrapolation scheme following Refs. [44, 45]. This scheme provides the populations in terms of their equilibrium and non-equilibrium components at the boundary via:

$$g_m(\mathbf{r}_b^\star, t^\star) = g_m^{\text{eq}}(p_b^\star, \mathbf{u}_b^\star) + \left(g_m(\mathbf{r}_f^\star, t) - g_m^{\text{eq}}(p_f^\star, \mathbf{u}_f^\star) \right), \quad (25)$$

where $\mathbf{r}_f^\star$ is one node to the interior of the boundary and $(p_f^\star, \mathbf{u}_f^\star)$ its corresponding pressure and velocity. For a Neumann-boundary condition at which we want to enforce $\partial_x \mathbf{u}^\star = \partial_x p^\star = 0$, for example, we can set $(p_b^\star, \mathbf{u}_b^\star) = (p_f^\star, \mathbf{u}_f^\star)$, such that the scheme reduces to prescribing populations for the unknown g_m at $\mathbf{r}_b^\star$ by copying them from $\mathbf{r}_f^\star$.

To illustrate how these boundary conditions can be incorporated into the streaming matrix, we consider a domain bounded by a wall region, an inlet region and outlet region, all with particular geometries. Every node is either an inlet node, an outlet node, a wall node, or otherwise a fluid node interior to the simulation region. We introduce the following indicator functions on the discrete phase space $\{(\mathbf{r}^\star, m)\}$ that encode the boundary regions within the model:

$$\mathcal{W}_{\mathbf{r}^\star} = \begin{cases} 1 & : \mathbf{r}^\star \text{ is a wall node,} \\ 0 & : \text{otherwise,} \end{cases} \quad (26a)$$

$$\mathcal{I}_{\mathbf{r}^\star, m} = \begin{cases} 1 & : \mathbf{r}^\star \text{ is an inlet node and } \mathbf{r}^\star + \mathbf{e}_m^\star \text{ is outside the simulation region,} \\ 0 & : \text{otherwise,} \end{cases} \quad (26b)$$

$$\mathcal{O}_{\mathbf{r}^\star, m} = \begin{cases} 1 & : \mathbf{r}^\star \text{ is an outlet node and } \mathbf{r}^\star + \mathbf{e}_m^\star \text{ is outside the simulation region,} \\ 0 & : \text{otherwise.} \end{cases} \quad (26c)$$

At the walls we impose a no-slip Dirichlet condition such that the effect of the first term in Eq. (24) can be achieved via the following modification of matrix elements of the streaming matrix:

$$S_{\text{wall}} = \sum_{\mathbf{r}^\star} \sum_{m=1}^Q (1 - \mathcal{W}_{\mathbf{r}^\star + \mathbf{e}_m^\star}) (1 - \mathcal{W}_{\mathbf{r}^\star}) |\mathbf{r}^\star + \mathbf{e}_m^\star\rangle \langle \mathbf{r}^\star| \otimes |m\rangle \langle m| \quad (27)$$

$$+ \sum_{\mathbf{r}^\star} \sum_{m=1}^Q \mathcal{W}_{\mathbf{r}^\star + \mathbf{e}_m^\star} (1 - \mathcal{W}_{\mathbf{r}^\star}) |\mathbf{r}^\star\rangle \langle \mathbf{r}^\star| \otimes |\bar{m}\rangle \langle m| \quad (28)$$

$$+ \sum_{\mathbf{r}^\star} \sum_{m=1}^Q \mathcal{W}_{\mathbf{r}^\star} |\mathbf{r}^\star\rangle \langle \mathbf{r}^\star| \otimes |m\rangle \langle m|. \quad (29)$$

In the above, the first term corresponds to unmodified streaming when both nodes considered are fluid; the second term encodes bouncing, when a particle in a fluid node hits a wall node; and the third one simply means no evolution (identity matrix) for the wall nodes. We note that the streaming matrix modified according to the above recipe is still a permutation matrix, so S_{wall} stays unitary and therefore can be realized directly by a quantum circuit.

At an outlet, we impose a Neumann-boundary condition. To account for this condition in the streaming matrix, we introduce one more correction of the following form:

$$S_{\text{Neumann}} = I + \sum_{\mathbf{r}^\star} \sum_{m=1}^Q \mathcal{O}_{\mathbf{r}^\star, m} |\mathbf{r}^\star\rangle \langle \mathcal{N}(\mathbf{r}^\star)| \otimes |\bar{m}\rangle \langle \bar{m}|, \quad (30)$$

and consider $S_{\text{Neumann}} S_{\text{wall}}$ instead of S_{wall} . This prescribes to simply replace the missing outlet occupations after streaming at position $\mathbf{r}^\star$ with a value copied from the neighboring interior node at position $\mathcal{N}(\mathbf{r}^\star)$, one lattice node away along the normal to the outlet towards the simulation region[†].

To model the edges of the simulation regions at an inlet and outlet, we also introduce the following correction term:

$$S_{\text{edge}} = \sum_{\mathbf{r}^\star} \sum_{m=1}^Q (1 - \mathcal{I}_{\mathbf{r}^\star, m}) (1 - \mathcal{O}_{\mathbf{r}^\star, m}) |\mathbf{r}^\star\rangle \langle \mathbf{r}^\star| \otimes |m\rangle \langle m|, \quad (31)$$

and consider $S_{\text{wall}} S_{\text{edge}}$ instead of S_{wall} . This term ensures that the fluid can go outside the simulation region via an inlet and an outlet, instead of moving periodically to the other side of the region. The final streaming term that accounts for walls, inlets and outlets is thus given by:

$$S' = S_{\text{Neumann}} S_{\text{wall}} S_{\text{edge}}. \quad (32)$$

[†]For example, in a two-dimensional setting with a vertical outlet being given by the points (N_x, y) with $y \in \{1, \dots, N_y\}$ and N_x the horizontal location of the outlet, we simply have $\mathcal{N}(\mathbf{r}^\star) = \mathbf{r}^\star - (1, 0)$.

This linear map is fully defined once we specify the indicator functions for the wall nodes, the inlet nodes, and the outlet nodes. If one can efficiently encode these three regions in a sufficiently compact functional description, then one can construct a block-encoding of S' , which governs the linearized dynamics of the fluid. For simplicity, we shall assume straight boundaries aligned with the lattice nodes, with inlet and outlet nodes forming orthogonal lines to the walls at either end of the desired simulation region.

2. Algorithmic extensions

As described in detail in Ref. [38], the workhorse of our quantum algorithm for LBE is the quantum linear solver of Ref. [46] applied to the linear system from Eq. (15). This solver takes as its inputs the state preparation unitary U_b and the unitary block-encoding U_A of the linear system A . Since the modifications to the streaming matrix described in the previous section changes A , we will now explain how to block-encode the required linear system in a way that incorporates the wall boundary and inlet/outlet physics.

Let us denote by A' the linear system using a modified streaming matrix S' , and by A the original system with periodic S . Let us also recall that with an appropriately chosen tensor encoding [38], the Carleman streaming matrices can be represented by $S = S^{\otimes N_C}$ and $S' = S'^{\otimes N_C}$. In Ref. [38], it was shown that the block-encoding prefactor α_A and the number of ancillary qubits needed n_A are given by:

$$\alpha_A = \alpha_C + 1, \quad n_A = n_C + 2, \quad (33)$$

where α_C and n_C are the block-encoding parameters for the Carleman collision matrix. This result was obtained under the assumption that S is unitary (so that $\alpha_S = 1$ and $n_S = 0$). However, it is straightforward to show that for a general matrix, such as S' , the expressions for the block-encoding parameters become:

$$\alpha_{A'} = \alpha_{S'}\alpha_C + 1, \quad n_A = n_{S'} + n_C + 2. \quad (34)$$

We can relate these parameters to the block-encoding parameters for the modified streaming term for a single-copy of the system. In particular, if we can construct a block-encoding of S' with block-encoding prefactor $\alpha_{S'}$ using $n_{S'}$ ancillary qubits, the linear system A' could be block-encoded with:

$$\alpha_{A'} = (\alpha_{S'})^{N_C} \alpha_C + 1, \quad n_{A'} = N_C n_{S'} + n_C + 2. \quad (35)$$

In order to achieve this, we will employ the following construction. First, recall that a partial permutation matrix is a matrix in which each row and each column contains at most one 1, and all other entries are 0. Given a partial permutation matrix $\tilde{\Pi}$, one can always complete it to a permutation matrix Π . These two matrices then satisfy:

$$\tilde{\Pi}\tilde{\Pi}^\dagger = \tilde{\Pi}\Pi^\dagger = \Pi\tilde{\Pi}^\dagger = \tilde{I}, \quad (36)$$

where $\tilde{I}$ is the identity matrix on the image of $\tilde{\Pi}$. One can then block-encode $\tilde{\Pi}$ with a unit block-encoding prefactor, $\alpha_{\tilde{\Pi}} = 1$, using a single ancillary qubit via:

$$U_{\tilde{\Pi}} = \tilde{\Pi} \otimes I + (\tilde{\Pi} - \Pi) \otimes X, \quad (37)$$

where we used a Pauli X unitary on the ancillary qubit. Indeed, using Eq. (36), one can verify that $U_{\tilde{\Pi}}$ is unitary and that

$$(I \otimes \langle 0|)U_{\tilde{\Pi}}(I \otimes |0\rangle) = \tilde{\Pi}. \quad (38)$$

Now, we observe that S_{edge} is a partial permutation matrix, whereas S_{Neumann} is a sum of two partial permutation matrices. Hence, using the block-encoding described above together with a linear combination of unitaries (LCU) [43] applied to two components of S_{Neumann} , we can straightforwardly construct the block encoding of S' with a block-encoding prefactor $\alpha_{S'} = 2$ using $n_{S'} = 3$ ancillary qubits. We thus conclude that:

$$\alpha_{A'} = 2^{N_C} \alpha_C + 1 \approx 2^{N_C} \alpha_A, \quad n_{A'} = n_A + 3N_C, \quad (39)$$

so that incorporating non-trivial geometries into our quantum algorithm for LBE increases the query complexity by a factor of 2^{N_C} and requires additional $3N_C$ ancillary qubits. This additional cost should not have a significant effect on the efficiency of the algorithm, as it is expected that relatively low N_C should suffice for simulations. More precisely, using the Carleman truncation error model [38], which predicts exponential damping of the Carleman error

as $\epsilon_C \propto 2^{-|\Gamma|N_C}$ (as long as one is within the convergence radius), the increase in the query complexity is scaling as $O(1/\epsilon_C^{1/|\Gamma|})$. For example, for the lid-driven cavity flow discussed in Sec. IV.B, one can estimate $|\Gamma|$ to be approximately 4.19 for $\text{Re} = 20$, which translates into an increase in the query complexity scaling as $O(1/\epsilon_C^{0.24})$.

Also, note that the complexity increase discussed above is due to a modified block-encoding prefactor, $\alpha_{A'}$ instead of α_A , but the query complexity can also increase due to the dependence on the condition number $\kappa_{A'}$, and not on κ_A . However, due to the fact that the difference between S' and S is only on the boundary nodes, this is expected to not be significant. The reasoning here is that this will depend on the norm $\|S' - S\|$, however this is a matrix of zeroes except for nodes on the boundary of the simulation region, and so will not scale with the volume of the region and should be sub-leading relative to other terms. As we shall see in Sec. IV.D, this is indeed confirmed by our numerical studies.

B. Driving

We now proceed to extending the model and quantum algorithm from Ref. [38] to accommodate flows driven by a divergence-free body force or a moving boundary.

1. Extending the model

In LBM, one can include the effect of a divergence-free body force $\mathbf{F}(\mathbf{r})$ by discretizing this force in velocity space, such that its discrete representation $\phi_m(\mathbf{r}^*)$, ensures that the zeroth moment respects mass conservation, $\sum_m \phi_m = 0$, and that the first moment recovers the forcing applied in the momentum equation, $\sum_m \mathbf{e}_m \phi_m = \mathbf{F}$. While a second-order accurate discretization (in space and time) can be obtained following Ref. [47], by adding ϕ_m to the right hand side of Eq. (9a) and including a velocity shift, we will instead pursue a first-order accurate approach as this does not require modifications of collision matrices F_1 and F_2 . As the implementation of a no-slip velocity boundary condition also includes a component which forces the flow (see Eq. (24)), we incorporate this driving which acts locally at $\mathbf{r}_b^*$ as:

$$g_m(\mathbf{r}_b^*, t) \rightarrow g_m(\mathbf{r}_b^*, t) + \phi_m(\mathbf{r}_b^*), \quad (40a)$$

such that $\phi_m(\mathbf{r}_b^*)$ in this case corresponds to $-2w_m \frac{\mathbf{e}_m^* \cdot \mathbf{u}_w^*}{c_s^{*2}}$, the correction of the bounce back rule for a moving boundary. Our matrix formulation of the problem from Eqs. (9a)-(9b) then gets extended by a driving step as follows:

$$\mathbf{g}^C(t^*) = (I + F_1) \mathbf{g}(t^*) + F_2 \mathbf{g}(t^*)^{\otimes 2}, \quad (41a)$$

$$\mathbf{g}^S(t^*) = S \mathbf{g}^C(t^*), \quad (41b)$$

$$\mathbf{g}(t^* + 1) = \mathbf{g}^S(t^*) + F_0, \quad (41c)$$

where F_0 is the driving vector given by:

$$F_0 = \sum_{\mathbf{r}^*} \sum_{m=1}^Q \phi_m(\mathbf{r}^*) |\mathbf{r}^*\rangle \otimes |m\rangle. \quad (42)$$

The effect that this third driving update rule from Eq. (41c) has on the Carleman vector can be inferred from its effect on k copies of the state vector:

$$[\mathbf{g}(t^* + 1)]^{\otimes k} = [\mathbf{g}^S(t^*) + F_0]^{\otimes k} = \sum_{l=0}^k [I^{\otimes l} \otimes F_0^{\otimes k-l} + \text{perms.}] \mathbf{g}^S(t^*)^{\otimes l} = \sum_{l=0}^k D_l^k \mathbf{g}^S(t^*)^{\otimes l}, \quad (43)$$

where we have introduced $d^k \times d^l$ matrices:

$$D_l^k := I^l \otimes F_0^{\otimes k-l} + \text{perms.}, \quad (44)$$

and where + perms. denotes a sum over distinct permutations over k subsystems, with l subsystems of type 1, and $k - l$ subsystems of type 2. This means that a single time step of the evolution can be compactly written as:

$$\mathbf{y}(t^* + 1) = \mathcal{D} S' C \mathbf{y}(t^*) + \mathcal{F}_0, \quad (45)$$

with the *Carleman driving matrix*:

$$\mathcal{D} = \begin{pmatrix} I & 0 & 0 & \dots & 0 \\ D_1^2 & I^{\otimes 2} & 0 & \dots & 0 \\ D_1^3 & D_2^3 & I^{\otimes 3} & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ D_1^{N_C} & D_2^{N_C} & D_3^{N_C} & \dots & I^{\otimes N_C} \end{pmatrix}, \quad (46)$$

and the *Carleman driving vector*:

$$\mathcal{F}_0 = [F_0, F_0^{\otimes 2}, \dots, F_0^{\otimes N_C}]^\top. \quad (47)$$

The linear system that captures the full evolution over N_t time steps, including non-trivial geometry encoded by $\mathcal{S}'$ and driving encoded by $\mathcal{D}$ and $\mathcal{F}_0$, is now given by:

$$\begin{pmatrix} I & 0 & 0 & \dots & 0 & 0 \\ -\mathcal{D}\mathcal{S}'C & I & 0 & \dots & 0 & 0 \\ 0 & -\mathcal{D}\mathcal{S}'C & I & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & I & 0 \\ 0 & 0 & 0 & \dots & -\mathcal{D}\mathcal{S}'C & I \end{pmatrix} \begin{pmatrix} \mathbf{y}(0) \\ \mathbf{y}(1) \\ \mathbf{y}(2) \\ \vdots \\ \mathbf{y}(N_t - 1) \\ \mathbf{y}(N_t) \end{pmatrix} = \begin{pmatrix} \mathbf{y}_{\text{ini}} \\ \mathcal{F}_0 \\ \mathcal{F}_0 \\ \vdots \\ \mathcal{F}_0 \\ \mathcal{F}_0 \end{pmatrix}, \quad (48)$$

and we will again write it compactly as $A''\mathbf{Y} = \mathbf{b}''$.

2. Algorithmic modifications

The algorithmic modifications needed to include driving are as follows. First, instead of block-encoding A' , we now need to block-encode A'' . And second, instead of constructing a state preparation unitary $U_{\mathbf{b}}$ for the undriven system, we need to construct $U_{\mathbf{b}''}$. In both cases, the central new object is the driving vector F_0 . Using the quantum data encoding proposed in Ref. [38], preparing F_0 is equivalent to block-encoding a square matrix $\bar{F}_0$ such that its first column is F_0 . Note that as long as $\|F_0\| \leq 1$, one can block-encode $\bar{F}_0$ with a unit prefactor using a single ancillary qubit. Moreover, the choice of LBE parameters proposed by Ref. [38], which guarantees that the norm of the LBE state vector is always bounded by 1, also ensures that the condition $\|F_0\| \leq 1$ is satisfied.

We start with constructing a block-encoding of A'' . As in the case of A' analyzed in the previous section, it is straightforward to show that the block-encoding parameters for A'' can be expressed via:

$$\alpha_{A''} = \alpha_{\mathcal{D}}\alpha_{\mathcal{S}'C} + 1, \quad n_A = n_{\mathcal{D}} + n_{\mathcal{S}'} + n_C + 2. \quad (49)$$

Hence, the actual task is to block-encode the Carleman driving matrix $\mathcal{D}$. For that, we will use some well-known block-encoding results [43]. First, employing linear combination of unitaries (LCU), if a matrix M can be written as:

$$M = \sum_{k=1}^K M_k, \quad (50)$$

where each M_k can be block-encoded with a prefactor α_{M_k} using n_{M_k} ancillary qubits, then one can construct a block-encoding of M with parameters:

$$\alpha_M = \sum_{k=1}^K \alpha_{M_k}, \quad n_M = \log K + \max_{k \in \{1, \dots, K\}} n_{M_k}. \quad (51)$$

A variation of LCU applicable to block-diagonal matrices of the form:

$$M = \sum_{k=1}^K |k\rangle\langle k| \otimes M_k \quad (52)$$

allows one to block-encode M with parameters:

$$\alpha_M = \max_{k \in \{1, \dots, K\}} \alpha_{M_k}, \quad n_M = 1 + \max_{k \in \{1, \dots, K\}} n_{M_k}. \quad (53)$$

Finally, for any $m \in \{1, \dots, N_C - 1\}$, the following N_C -dimensional shift matrix,

$$\Delta_m = \sum_{k=m+1}^{N_C} |k\rangle\langle k-m|, \quad (54)$$

can be block-encoded with a unit prefactor using a single ancillary qubit, as it is a partial permutation matrix.

Now, let us write $\mathcal{D}$ explicitly, using the encoding with the first register keeping track of the Carleman order:

$$\mathcal{D} = \sum_{k=1}^{N_C} \sum_{l=1}^k |k\rangle\langle l| \otimes D_l^k = \sum_{k=1}^{N_C} \sum_{m=0}^{k-1} |k\rangle\langle k-m| \otimes D_{k-m}^k = \sum_{m=0}^{N_C-1} \sum_{k=m+1}^{N_C} (|k\rangle\langle k| \otimes D_{k-m}^k) (\Delta_m \otimes I) =: \sum_{m=0}^{N_C-1} B_m. \quad (55)$$

Using Eq. (51), we then have:

$$\alpha_{\mathcal{D}} = \sum_{m=0}^{N_C-1} \alpha_{B_m}, \quad n_{\mathcal{D}} = \log N_C + \max_{m \in \{0, \dots, N_C-1\}} n_{B_m}, \quad (56)$$

and the problem reduces to constructing block-encodings of B_m matrices. To achieve this, we can block-encode the two factors of B_m separately. As already mentioned, the second factor, $\Delta_m \otimes I$, can be block encoded with a unit prefactor and one ancillary qubit, whereas for the first factor, we will employ Eq. (53). This results in the following block-encoding parameters for B_m :

$$\alpha_{B_m} = \max_{k \in \{m+1, \dots, N_C\}} \alpha_{D_{k-m}^k}, \quad n_{B_m} = 2 + \max_{k \in \{m+1, \dots, N_C\}} n_{D_{k-m}^k}. \quad (57)$$

Finally, recalling Eq. (44), we note that each D_{k-m}^k consists of a sum over $\binom{k}{m}$ terms, each containing m copies of $\bar{F}_0$. Hence, using Eq. (51), we have:

$$\alpha_{D_{k-m}^k} = \binom{k}{m}, \quad n_{D_{k-m}^k} = \log \binom{k}{m} + m. \quad (58)$$

Putting it all together, we arrive at:

$$\alpha_{\mathcal{D}} = \sum_{m=0}^{N_C-1} \binom{N_C}{m} = 2^{N_C} - 1, \quad (59a)$$

$$n_{\mathcal{D}} = \log N_C + 2 + \max_{m \in \{0, \dots, N_C-1\}} \left(\log \binom{N_C}{m} + m \right) \leq \log N_C + 2N_C + 1. \quad (59b)$$

Substituting the expressions derived above, together with the previous results from Eq. (39), into Eq. (49), we thus conclude that:

$$\alpha_{A''} = (2^{N_C} - 1)2^{N_C} \alpha_C + 1 \approx 4^{N_C} \alpha_A, \quad n_{A''} = n_A + 5N_C + \log N_C + 1. \quad (60)$$

Thus, the complexity cost related to extending the algorithm from Ref. [38] to include both non-trivial geometries and forcing increases the query complexity by a factor of 4^{N_C} , and requires additional $5N_C$ ancillary qubits (up to leading order in N_C). As already discussed in the previous section, this cost should not strongly affect the algorithm's efficiency, as N_C can be chosen to be small in the sub-threshold regime, due to exponential convergence of the Carleman expansion. However, there are two other sources of potential complexity increase: the difference between the condition numbers of A and A'' , and the cost related to state preparation unitary $U_{b''}$ instead of U_b . We postpone the discussion of the first effect to Sec. IV.D, and now focus on constructing $U_{b''}$.

Let us first assume that one can efficiently construct a unitary U_{F_0} that prepares a quantum state $|F_0\rangle$ that is a normalized version of the driving vector F_0 from Eq. (42). Then, as was shown in Ref. [28], one can prepare $|\mathcal{F}_0\rangle$ using $O(N_C)$ queries to U_{F_0} and $O(N_C)$ single qubit gates. Hence, each query to a state preparation unitary $U_{\mathcal{F}_0}$ translates into $O(N_C)$ queries to U_{F_0} . Next, for simplicity, let us assume that one initializes each node of the fluid system in an

equilibrium state corresponding to zero velocity and zero pressure (as is typical in CFD simulations, and as we do in all examples presented in Sec. IV). This then means that $\mathbf{y}_{\text{ini}}$ is a zero vector. Additionally, adding a single driving step before the start of the evolution, we can rewrite $\mathbf{b}''$ as

$$\mathbf{b}'' = \sum_{t^*=0}^{N_t} |t^*\rangle \otimes \mathcal{F}_0. \quad (61)$$

Hence, preparing $|\mathbf{b}''\rangle$ just requires a uniform state preparation on the time step register and a single preparation of $|\mathcal{F}_0\rangle$, which in turn requires $O(N_C)$ uses of U_{F_0} . Therefore, one can conclude that, as long as one can efficiently prepare $|\mathcal{F}_0\rangle$, one can also efficiently prepare $|\mathbf{b}''\rangle$.

Although the complexity cost of preparing $|\mathcal{F}_0\rangle$ may be significant for random or unstructured driving, in practice one is usually interested in driving with explicit functional dependence on position. For example, in the lid-driven cavity flow and flow past a cylinder, analyzed in the next section, driving has a particularly simple form:

$$F_0 = \sum_{\mathbf{r}^* \in \mathcal{R}} |\mathbf{r}^*\rangle \otimes \sum_{m=1}^Q \phi_m |m\rangle. \quad (62)$$

In the above, $\mathcal{R}$ is a subset of nodes (neighboring the lid and inlet ones, respectively) and ϕ_m are just a constant driving coefficients encoded in a small velocity register (consisting of $2D$ qubits for fluid in D dimensions). Hence, we just need a uniform state preparation on the position register and a general state preparation on a small velocity register. The complexity cost of preparing $|\mathcal{F}_0\rangle$ is thus constant. Therefore, we will assume that the complexity cost of $U_{\mathbf{b}''}$ is negligible compared to the cost of unitary block-encoding $U_{A''}$ of the linear system.

IV. Numerical studies

In Sec. II, we introduced a quantum algorithm based on a Carleman embedding of the incompressible lattice Boltzmann method as described in Ref. [38]. In the previous Section III, we demonstrated how this approach can be extended to model non-trivial flows in domains which do not have periodic boundary conditions. In particular, we showed how: no-slip boundary conditions, free-slip boundary conditions, body forcing and non-trivial geometries can be accommodated. In this section, we implement and numerically test these theoretical developments in order to assess the ability of the Carleman embedding to approximate three different flows demanding these extensions.

We focus on the driven Taylor-Green vortex, lid-driven cavity flow, and the flow past a cylinder, enclosed within a simulation region of size $L_x \times L_y$. They are described by the incompressible Navier-Stokes equations:

$$\partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho_0} \nabla p + \nu \nabla^2 \mathbf{u}, \quad (63a)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (63b)$$

together with an appropriate set of boundary conditions. In each case, the forcing applied (via the boundary conditions or via a body forcing $\mathbf{F}$), has an associated characteristic velocity-scale U and length-scale L , which allows a physical Reynolds number $\text{Re} = UL/\nu$ to be constructed. For the lid-driven cavity flow, for example, these scales correspond to the speed of the wall and the cavity size, while for the flow past a cylinder they correspond to the inlet velocity and the cylinder diameter. However, given that we numerically solve the lattice Boltzmann equation, Eq. (5), which is non-dimensionalized in terms of lattice units (i.e., using a velocity scale $\Delta x/\Delta t$ and length-scale Δx), it is useful to first establish the relationship between Re and these lattice units. As described in Ref. [38], guided by turbulence theory and the scaling required to maintain $\|\mathbf{g}\| \sim O(1)$, we set:

$$\text{Ma} \approx \frac{u_0^*}{c_s^* \sqrt{N_x N_y}}, \quad (64)$$

to obtain this relation. The dimensionless parameter u_0^* introduced as a control of the temporal resolution is set to $u_0^* = 1$ unless otherwise stated. For a periodic box, the number of grid-points per direction required to resolve homogeneous isotropic turbulence scales as $N_x \gtrsim \eta_x \text{Re}^{3/4}$ and $N_y \gtrsim \eta_y \text{Re}^{3/4}$, where $\eta_i = L_i/L$ (see Ref. [48]). On this basis, we fix $N_i = \eta_i \text{Re}^\beta$ with $\beta \geq 3/4$, while for the temporal resolution we make use of the advection time-scale $T = L/U$,

Re	$N_C = 1$		$N_C = 2$		$N_C = 3$	
	d_C	$d_C(N_t + 1)$	d_C	$d_C(N_t + 1)$	d_C	$d_C(N_t + 1)$
10	$9 \cdot 10^2$	$9 \cdot 10^4$	$8.1 \cdot 10^5$	$8.1 \cdot 10^7$	$7.3 \cdot 10^8$	$7.3 \cdot 10^{10}$
100	$9 \cdot 10^4$	$9 \cdot 10^8$	$8.1 \cdot 10^9$	$8.1 \cdot 10^{13}$	$7.3 \cdot 10^{14}$	$7.3 \cdot 10^{18}$
1000	$9 \cdot 10^6$	$9 \cdot 10^{12}$	$8.1 \cdot 10^{13}$	$8.1 \cdot 10^{19}$	$7.3 \cdot 10^{20}$	$7.3 \cdot 10^{26}$

Table 1 Dimensions d_C of the Carleman vector and $d_C(N_t + 1)$ of the Carleman linear system for two-dimensional LBE problems with resolution parameter $\beta = 1$, different Reynolds numbers Re and truncation orders N_C .

which non-dimensionally translates into the number of time-steps $N_t = \frac{1}{\Delta t} \frac{L}{U}$. Using the definition of the Mach number $\text{Ma} = U/(c_s^* \frac{\Delta x}{\Delta t})$, together with Eq. (64), we can eliminate Δt giving $N_t = N_x N_y / (u_0^* \sqrt{\eta_x \eta_y})$. For a given simulation with fixed η_x and η_y , it therefore suffices to specify Re and β , as this fixes the remaining parameters according to:

$$N_x = \lceil \eta_x \text{Re}^\beta \rceil, \quad N_y = \lceil \eta_y \text{Re}^\beta \rceil, \quad N_t = \lceil N_x N_y / (u_0^* \sqrt{\eta_x \eta_y}) \rceil, \quad \tau^* = \frac{3u_0^*}{\sqrt{\eta_x \eta_y}} \frac{1}{\text{Re}}. \quad (65)$$

Varying β and u_0^* then allows one to control the spatial and temporal resolution if needed.

Our numerical implementation of the classical LBM and Carleman LBM is programmed in such a way that linear operations including streaming (which incorporates the boundary conditions) and forcing are represented as matrices. These matrices are then used by both our classical and Carleman LBM routines when performing these operations. This construction was intentionally chosen, as by validating our classical LBM code, we in-turn validate all linear components of the Carleman LBM. We performed validation tests by correctly reproducing the time evolution of the decaying and forced Taylor-Green vortex with second-order error in both space $O(\Delta x^2)$ and time $O(\Delta t^2)$ (see Fig. 8 in the Appendix). Given that the Taylor-Green vortex does not test the validity of our implementation of the no-slip and slip-free boundary conditions, we also performed our simulations with a second independent code, which we used to verify our implementation.

To assess the ability of the Carleman LBM to correctly reproduce the classical LBM, we use the relative L_2 error:

$$\epsilon_{\text{rel}}(t^*) := \frac{\|\mathbf{u}^{N_C}(\mathbf{r}^*, t^*) - \mathbf{u}(\mathbf{r}^*, t^*)\|}{\|\mathbf{u}(\mathbf{r}^*, t^*)\|}, \quad \epsilon_C := \max_{t^* \in \{1, \dots, N_t\}} \epsilon_{\text{rel}}(t^*), \quad (66)$$

where $\|\mathbf{u}(\mathbf{r}^*, t^*)\|^2 = 1/V \int |\mathbf{u}|^2 dV$ and superscript N_C indicates the Carleman LBM solution with truncation order N_C . Note that only $t^* > 0$ is considered, as any error at $t^* = 0$ is independent of the embedding method. Although it would, in principle, be desirable to fix $\text{Re} \gg 1$ and investigate how ϵ_C depends on the Carleman truncation order N_C , the dimension of the linear embedding from Eq. (45) (as illustrated by Table 1) grows with N_C according to Eq. (11), thus making this calculation extremely demanding for classical computers. Nevertheless, in order to at least attempt to accurately simulate physically relevant configurations, we must choose sufficiently large $N_x = \eta_x \text{Re}^\beta$, which in turn restricts our calculations to $N_C = 1, 2, 3$. Although this is compounded by the fact our code is currently limited to serial runs, we envisage making use of GPUs in future studies.

Having detailed our numerical implementation, we now consider our three examples: how ϵ_C depends on Re and N_C , how the spatial form of the absolute value of the difference $|\mathbf{u}^{N_C}(\mathbf{r}^*, t^*) - \mathbf{u}(\mathbf{r}^*, t^*)|$ varies with N_C , and how $\epsilon_{\text{rel}}(t^*)$ varies in time. We consider the driven Taylor-Green vortex first as it is the simplest case, followed by the lid-driven cavity, and the flow past a cylinder. Finally, we will present numerical results on the condition number of the associated linear systems, thus addressing the question about the complexity cost of simulating non-trivial flows.

A. Driven Taylor-Green vortex

In a 2D domain $\Omega := [0, L] \times [0, L]$ with periodic boundary conditions, the Taylor-Green vortex state is given by:

$$\mathbf{u}(\mathbf{r}, t) = [\sin(kx) \cos(ky), -\cos(kx) \sin(ky)] f(t), \quad p(\mathbf{r}, t) = -\frac{\rho_0}{2} (\sin^2(kx) + \sin^2(ky)) f^2(t), \quad (67)$$

where $\mathbf{r} = (x, y)$, the wavenumber $k = 2\pi/L$, and $f(t)$ is explicit time dependence. Substituted into the Navier-Stokes equations, Eqs. (63a)-(63b), this state leads to the cancellation of the pressure gradient and non-linear terms, leaving

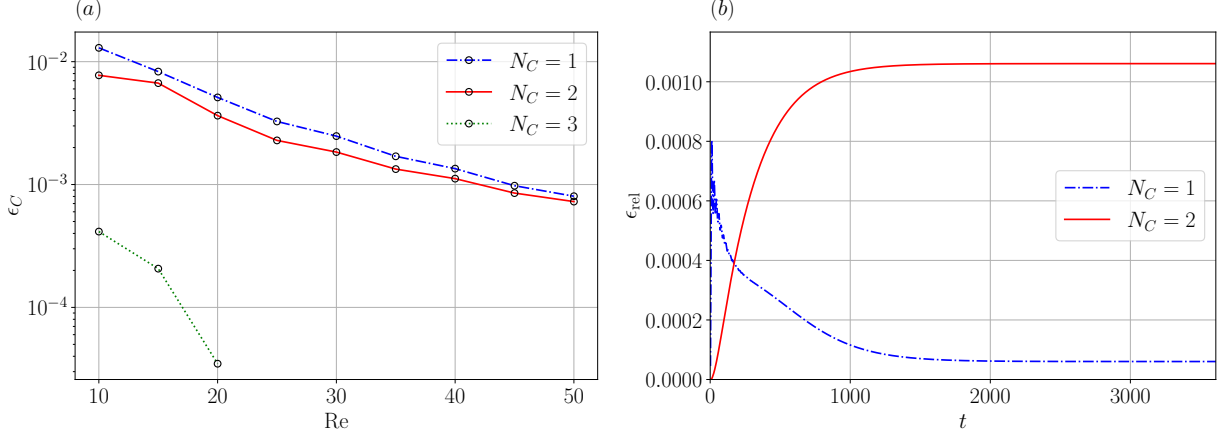


Fig. 1 Carleman truncation error for the forced Taylor-Green vortex flow with $\beta = 0.75$. **Left:** Dependence of ϵ_C on the Reynolds number Re and truncation order $N_C = 1, 2, 3$ for one advection time. **Right:** Time evolution of ϵ_{rel} over 10 advection times for $N_C = 1, 2$ and $Re = 50$.

$\partial_t \mathbf{u} = \nu \Delta \mathbf{u}$, which when solved with the initial condition $f(t = 0) = U_{TG}$ gives $f(t) = U_{TG} \exp(-2\nu k^2 t)$. As this solution decays exponentially in time, it is interesting to determine the form of a forcing $\mathbf{F}$, which when added to the right hand side the Navier-Stokes equations not only sustains Eq. (67), but also leaves open the possibility of producing time-dependent states as the forcing increases in strength. Setting $\partial_t \mathbf{u} = 0$ and substituting Eq. (67) into these equations implies a forcing of the form:

$$\mathbf{F} = 2\nu k^2 \mathbf{u}(\mathbf{r}, t = 0). \quad (68)$$

Choosing the velocity scale U_{TG} and length-scale L , we can non-dimensionalize the problem such that $Re = U_{TG} L / \nu$ characterizes the relative strength of this forcing. In order to test the extension of the Carleman LBM to include driving as described in Sec. III.B, we follow the approach of Ref. [47] that considered the addition of a divergence-free body force for the incompressible LBM [39]. For a steady or slowly varying flow such that $\partial_t \mathbf{u} \approx 0$, it is consistent to add

$$\phi_m = w_m \frac{\mathbf{e}_m \cdot \mathbf{F}}{c_s^2} \quad (69)$$

to the right hand side of Eq. (5). As Eq. (67) is a steady solution of the forced problem, we validated our implementation of the driving is *nearly* second-order accurate[‡] as shown in Fig. 8 in the Appendix. However, in order to conduct a more strenuous test of the Carleman convergence, we choose instead to investigate transient dynamics, by starting from the zero initial condition $\mathbf{u}(\mathbf{r}, t = 0) = 0$, $p(\mathbf{r}, t = 0) = 0$ and tracking how the classical and Carleman LBM evolve to the steady state solution from Eq. (67).

Figure 1(a) shows the convergence behavior of the Carleman error ϵ_C for $N_C = 1, 2, 3$ with $\beta = 0.75$. We observe that the error for the linear Carlemen LBM, $N_C = 1$, decreases with increasing Re . This was also observed for the decaying Taylor-Green vortex [38], and may be attributed to the fact that as $N_x = Re^\beta$ increases, the Mach number reduces in accordance with Eq. (64), thereby reducing the strength of nonlinearity. Although the quadratic Carleman expansion does not significantly improve the relative error, by continuing to $N_C = 3$ we do find a dramatic reduction. This demonstrates that at least for small Re the proposed Carleman LBM for forced flows can converge as N_C increases, albeit not necessarily uniformly. Due to the need to consider $N_C = 3$ to observe a convincing trend towards convergence, it was necessary to limit our spatial resolution to $\beta = 0.75$. However, by recomputing Fig. 1(a) for $\beta = 1$ (without $N_C = 3$ included), we verified that the results presented are consistent with those observed for a larger resolution.

To assess the time evolution of the truncation error, we chose $Re = 50$ (corresponding to $N_x = N_y = 19$) and time-integrated the zero initial condition subject to the Taylor-Green forcing for 10 advection times. As shown in Fig. 1(b), the relative error ϵ_{rel} plateaus following an initial transient. This shows that error associated with the Carleman LBM for forced flows does not continually grow, but converges to a finite value. The spatial form of the solution and its associated Carleman truncation error are shown in Fig. 2. Although it is interesting to go to larger Re , we

[‡]The velocity shift proposed in Ref. [47] requires modifications of F_1 and F_2 which are not currently included in our code.

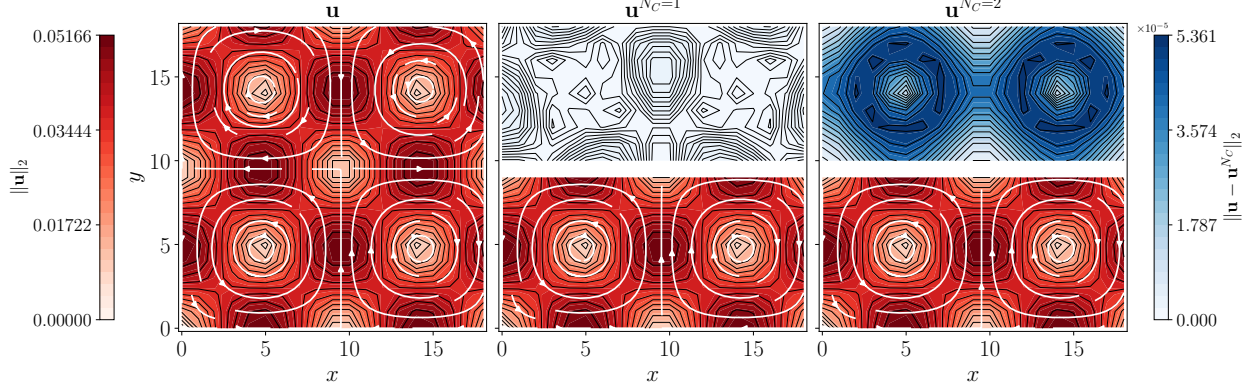


Fig. 2 Spatial form of the classical LBM solution, the Carleman LBM solution and their associated error for the forced Taylor-Green vortex flow after 10 advection times for $\beta = 0.75$ and $\text{Re} = 50$. **Left:** Classical LBM solution. **Middle:** Linear Carleman approximation (bottom) and error (top). **Right:** Quadratic Carleman approximation (bottom) and error (top).

intentionally limit the study to $\text{Re} = 50$, as beyond this threshold the solution state shown in Fig. 2 becomes unstable and can transition to another more complex steady state. Since it is known that a single, global linearization scheme cannot describe multiple basins of attraction, it follows that a direct Carleman embedding will fail for Reynolds number above a certain value. However, recent work has begun to go beyond such obstacles by constructing adaptive linearization schemes [49]. It would be of interest to extend such methods to quantum algorithms, and the above driven Taylor-Green vortex could be an excellent test-bed for such methods.

B. Lid-driven cavity

To test the extension of the Carleman LBM for no-slip boundary conditions as described in Sec. III, we consider lid-driven cavity flow, initially studied for the classical incompressible LBM in Ref. [40]. This problem considers a 2D cavity, for example a notch of size $\Omega := [0, H] \times [0, H]$ taken out of an aeroplane wing, such that the air flowing over the top surface moves at constant velocity U_w , causing the sheared fluid within the cavity to circulate. Letting $\mathbf{u} = (u, v)$, we set the components of the velocity vector at the top $y = H$ and bottom $y = 0$ boundaries of the cavity to

$$u(x, y = H, t) = U_w, \quad v(x, y = H, t) = 0, \quad u(x, y = 0, t) = 0, \quad v(x, y = 0, t) = 0, \quad (70)$$

and at the sides to

$$\mathbf{u}(x = 0, y, t) = \mathbf{u}(x = H, y, t) = 0. \quad (71)$$

As we use the lattice Boltzmann method, these macroscopic boundary conditions are converted into rules for how the populations g_m are updated at the walls as described in Secs. III.A-III.B. By choosing the velocity scale U_w and the box size as a length-scale H , we can non-dimensionalize the problem such that $\text{Re} = U_w H / \nu$ characterizes the strength at which the moving top boundary drives the flow. As per the Taylor-Green vortex, we start from the zero initial condition $\mathbf{u}(\mathbf{r}, t = 0) = 0$, $p(\mathbf{r}, t = 0) = 0$ and track how the classical and Carleman LBM evolve to a steady state solution.

Figure 3(a) shows the convergence behavior of the Carleman error for $N_C = 1, 2$. In contrast to the previous example, we observe that as Re increases, the error grows for both $N_C = 1$ and $N_C = 2$. Also, by going to $N_C = 2$, we directly observe a big drop in the error ϵ_C which appears to shrink as Re increases. On this basis, we did not go to $N_C = 3$, but instead increased the spatial resolution for this example by setting $\beta = 1$, such that $N_x = N_y = \text{Re}$. We attribute the increase in ϵ_C with Re to the fact that the form of the solution in this case changes as Re increases, and in particular that strong velocity gradients develop near the lid as shown in Fig. 4. For the Taylor-Green vortex, by contrast, the Carleman embedding only needed to approximate the same form of solution (given by Eq. (67)) for all Re . This is because for the range of Re considered, only the time-scale at which we reach the steady state changes with Re .

Figure 3(b) shows the time evolution of the relative error for $N_C = 1, 2$ at $\text{Re} = 25$. Once again, we observe that the Carleman error does not continually increase in time but instead converges to a finite value. A second interesting observation is that oscillations appear in ϵ_{rel} for both $N_C = 1$ and $N_C = 2$, which decay as t increases. However, the amplitude of these oscillation is much larger for $N_C = 1$. Similar behavior was also observed for the Taylor-Green

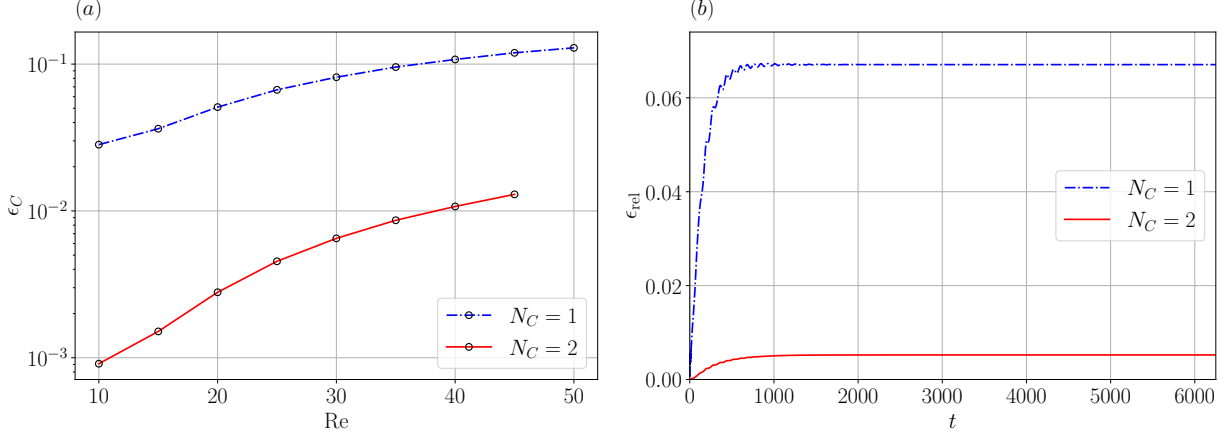


Fig. 3 Carleman truncation error for the lid-driven cavity flow with $\beta = 1$. **Left:** Dependence of ϵ_C on the Reynolds number Re and truncation order $N_C = 1, 2$ for one advection time. **Right:** Time evolution of ϵ_{rel} over 10 advection times for $N_C = 1, 2$ and $Re = 25$.

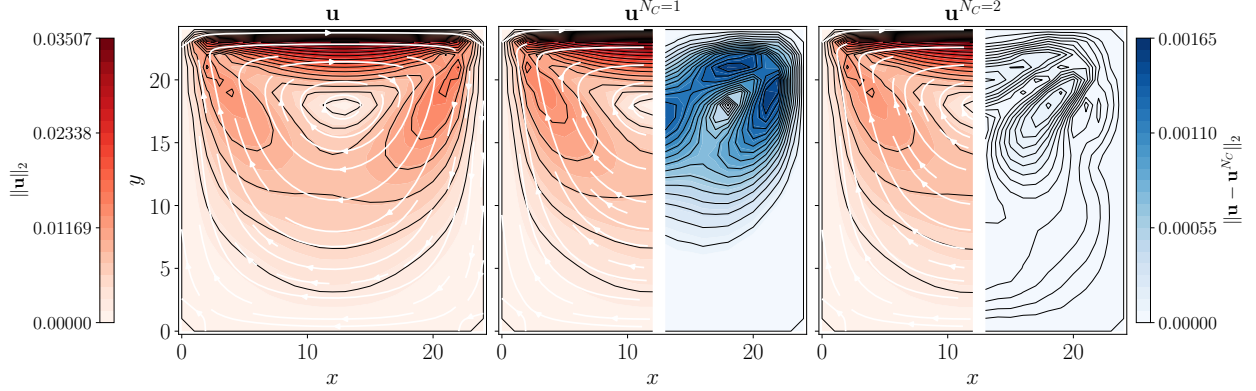


Fig. 4 Spatial form of the classical LBM and Carleman LBM solutions (red) and their associated error (blue) for the lid-driven cavity flow after 10 advection times for $\beta = 1$ and $Re = 25$. **Left:** the classical LBM solution. **Middle:** linear Carleman approximation. **Right:** the quadratic Carleman approximation.

vortex in Fig. 1(b). Taken together, the results from Fig. 3 demonstrate that at least for small Re the proposed Carleman LBM for shear flows with no-slip boundary conditions converges as N_C increases.

Figure 4 shows the spatial form of the solution predicted by the classical LBM as well as the linear $u^{N_C=1}$ and quadratic $u^{N_C=2}$ Carleman LBM approximations. Due to horizontal movement of the lid, we observe larger gradients of the velocity near the moving lid. As the strongest gradients develop near the top right hand corner, the location where the fluid traveling rightwards meets a solid boundary and is forced to re-circulate, we have chosen to highlight the form of spatial error in this region. Comparing the form of the error for the linear $u^{N_C=1}$ and quadratic $u^{N_C=2}$ approximations, we observe that $N_C = 2$ further reduces the intensity of the error and shifts it away from the walls. In both solutions, we also observe that exactly at the boundaries the error is smallest. This suggests that the proposed extension of the Carleman LBM for no-slip boundary conditions yields a physically consistent approximation.

C. Flow past a cylinder

In order to computationally model an object in a wind tunnel, it is necessary to be able to accommodate inlet/outlet flows. A simple example in which the flow enters the domain, flows around the object embedded within, and then exits the domain, is the flow around a cylinder. Following Ref. [40] that studied this flow for the classical incompressible LBM, we use this example to test the extension of the Carleman LBM described in Section III to include free-slip

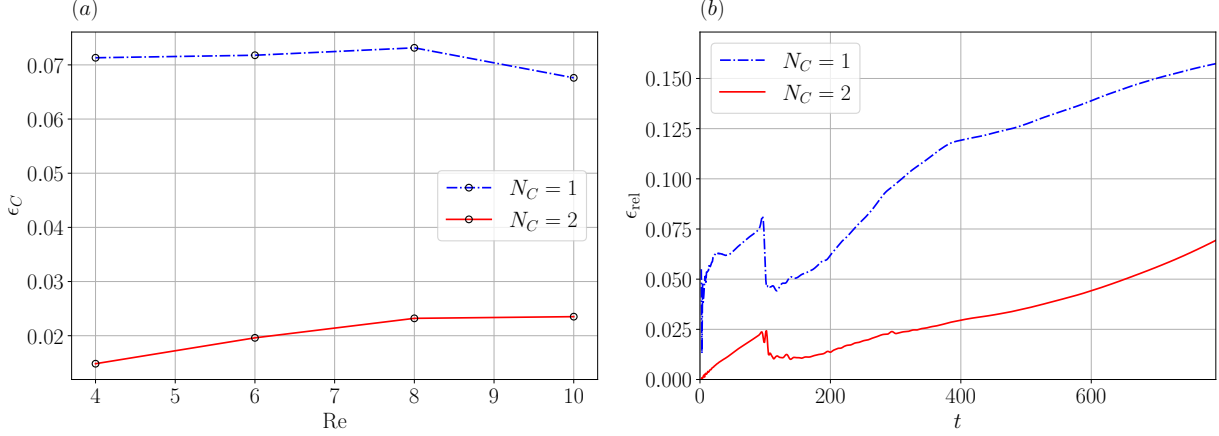


Fig. 5 Carleman truncation error for the flow past a cylinder with $\beta = 0.75$. **Left:** Dependence of ϵ_C on the Reynolds number Re and truncation order $N_C = 1, 2$ for one advection time. **Right:** Time evolution of ϵ_{rel} over 5 advection times for $N_C = 1, 2$ and $Re = 6$.

outflow boundary conditions, as well as no-slip boundary conditions on a cylinder embedded within the domain.

The computational domain considered is $\Omega := [-2.5D, 11.5D] \times [-3.5D, 3.5D]$ where D is the cylinder diameter. The cylinder is positioned at $\mathbf{r} = (0, 0)$, slightly downstream of the inlet, but sufficiently far from the outlet, so that the influence of the outlet boundary condition is not felt upstream. In the vertical, we use periodic boundaries, at the inlet we impose the no-slip boundary condition

$$\mathbf{u} = (U_\infty, 0), \quad (72)$$

according to Eq. (24), while at the outlet we impose

$$\partial_x u = \partial_x v = 0, \quad (73)$$

such that the flow is free to exit the right hand side of the domain with a vertically non-uniform velocity profile. To convert the latter boundary condition into rules for how the populations g_m should be updated at the outlet $\mathbf{r}_o$, we use non-equilibrium extrapolation scheme to implement a zero-gradient (pressure and velocity) outlet[§] following Refs. [44, 45], whose implementation is described in Sec. III.A. Due to the large domain required to minimize the influence of the outflow boundary on the flow, we are limited to small $Re \leq 10$ and $N_C = 1, 2$ with $\beta = 0.75$, where the Reynolds number is defined by $Re = DU_\infty/\nu$. As before, we initialize the systems at zero initial condition with $\mathbf{u}(\mathbf{r}, t = 0) = (0, 0)$, $p(\mathbf{r}, t = 0) = 0$.

Figure 5(a) shows the Carleman truncation error for $N_C = 1, 2$ as a function of Re . Due to the small variations in spatial resolution with Re , we do not observe large differences in ϵ_C for this case. Consistent with the previous two cases considered, however, we do find that by increasing the Carleman truncation order from $N_C = 1$ to $N_C = 2$ we can reduce the maximum relative error over one advection time from 7% to 2.5%. Consistent with the lid-driven cavity flow, we observe that over 5 advection times the relative error for $N_C = 2$ remains less than $N_C = 1$. The dynamics of the relative error for this case is, however, more complex. In particular, at $t \approx 100$ we observe a drop in the relative error after an initial increase. We attribute this signature of the error signal to be the time taken for the wake of the cylinder to reach the outlet boundary. Following this decrease, the relative error subsequently increases for both cases, saturating around 15% after approximately 10 advection times for $N_C = 1$. For $N_C = 2$, we have not been able to run the simulate for long enough to establish this saturation but envisage using a more efficient code implementation in the future to establish this result.

The top frame of Fig. 6 shows the classical solution obtained for this problem when $Re = 6$ and $\beta = 0.75$. Due to the computational constraints imposed by the Carleman LBM calculation, we use $N_x \times N_y = 58 \times 26$ lattice nodes, the cylinder is coarsely represented using $D = 4$ (corresponding to 16 lattice nodes), and we are restricted to steady state solutions. In this regime we observe higher velocities/kinetic energy as darker red above and below the cylinder. Behind

[§]We acknowledge that a fixed pressure outlet with zero-gradient velocity handles reflections due to the finite domain better and has improved stability. Future work will consider this modification of the outflow boundary condition.

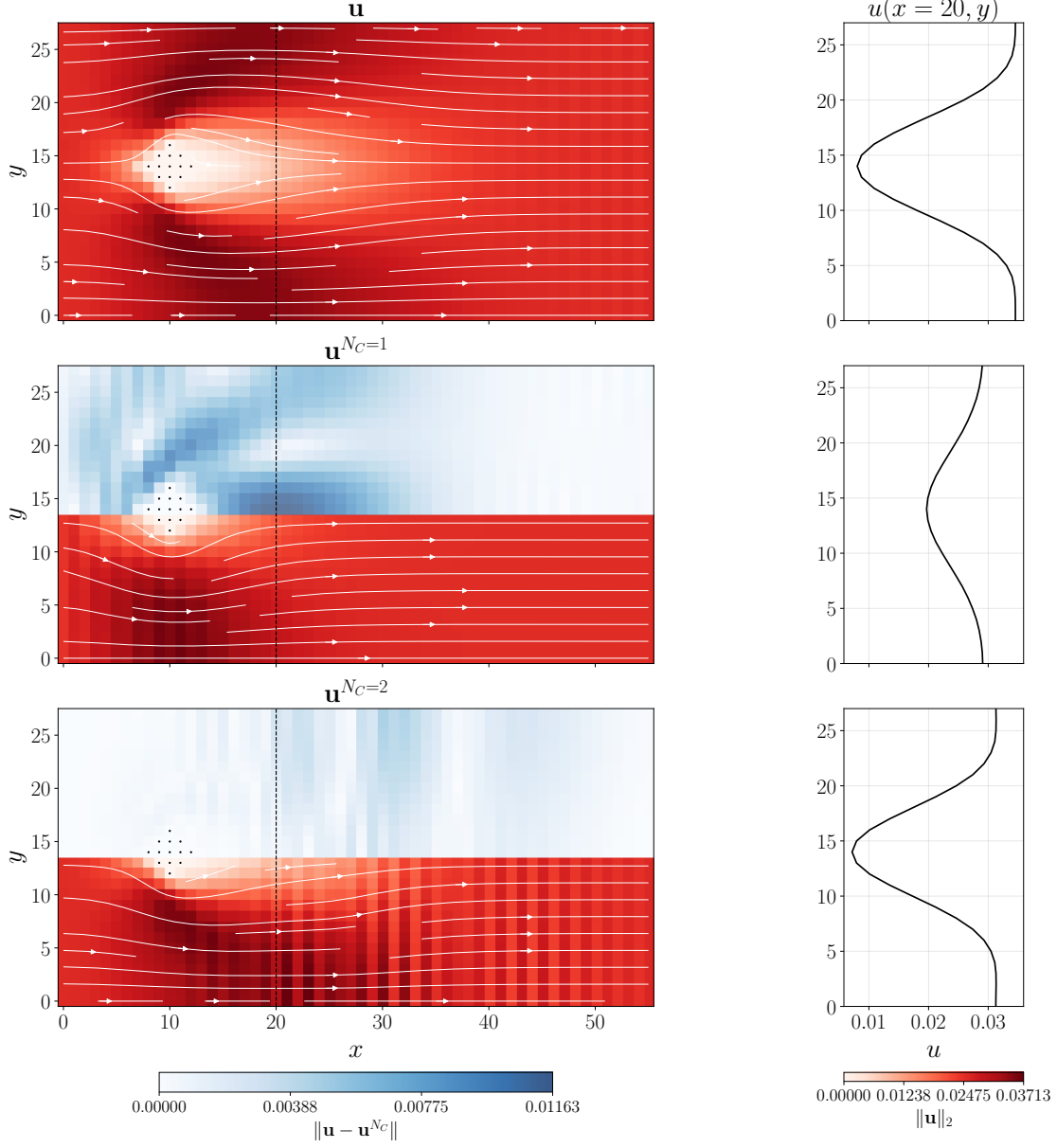


Fig. 6 Spatial form of the classical LBM and Carleman LBM solutions (red) and their associated error (blue) for the flow around a cylinder after 5 advection times for $\beta = 0.75$ and $\text{Re} = 6$. Left: The classical LBM solution accompanied by its streamwise velocity profile (top), the linear Carleman LBM approximation (red, middle) and the quadratic Carleman LBM approximation (red, bottom). The Carleman LBM truncation error is shown in blue in the middle and bottom frames. Right: Horizontal velocity profile evaluated downstream of the cylinder at the location indicated by the dashed line.

the cylinder, the lower kinetic energy (lighter red) reflects the velocity deficit caused by the cylinder. The vertical profile of the horizontal velocity evaluated downstream of the cylinder, shown on the top right of Fig. 6, also reflects this velocity deficit.

The linear Carleman approximation (red) accompanied by its deviation from the classical LBM solution (blue) is shown in the middle of Fig. 6. We observe that for the linear approximation the deviation is located primarily above and below the cylinder, where the increase in kinetic energy is misrepresented, as well as behind the cylinder, where the velocity deficit is misrepresented. The misrepresentation of the velocity deficit is emphasized in the deviation of

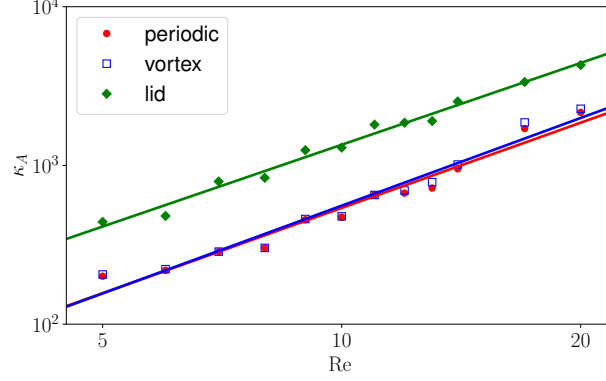


Fig. 7 Condition number κ_A as a function of the Reynolds number Re for the periodic system, driven Taylor-Green vortex, and lid-driven cavity with parameters $\beta = 0.75$ and $N_C = 2$. Points correspond to numerical values and solid lines to best fits as described by Eq. (74).

$u(x = 20, y)$, for $N_C = 1$, from the classical simulation. Increasing the Carleman truncation order to $N_C = 2$ (see bottom of Fig. 6), we find that the misrepresentation of the classical LBM solution above/below and behind the cylinder is drastically reduced. This is also seen when comparing the profiles for $u(x = 20, y)$ in the bottom and top right frames of Fig. 6. By contrast, we observe that $N_C = 2$ does not recover this velocity deficit without error, and in particular waves generated in the wake of the cylinder are found to be reflected from the outflow boundary in this case. Due to computational constraints it has not yet been possible to test $N_C = 3$ in order to establish whether this is an error specific to $N_C = 2$.

D. Condition number

As already mentioned in Sec. III, the extensions to non-trivial geometries and driven flows introduced in this paper may bring additional complexity cost as compared to the original algorithm from Ref. [38], as these modifications may affect the condition number of the linear system from Eq. (48). This, in principle, might negate the potential for a quantum advantage reported in Ref. [38]. However, the numerical results that we will now present paint an optimistic picture, in which this extra cost is small and does not affect the asymptotic scaling of the algorithm's query complexity.

Due to the immense size of the linear system whose condition number we want to investigate (recall Table 1), we limit our numerical studies to $\beta = 0.75$, $Re \in [5, 20]$, $N_C = 1, 2$, and do not consider the flow past a cylinder case. Using a Lanczos-based algorithm, we first numerically compute the condition number $\kappa_{A_{\text{per}}}$ of the linear system describing undriven and periodic LBE, i.e., A_{per} is given by Eq. (48) with Carleman driving $\mathcal{D}$ replaced by an identity matrix and Carleman streaming $\mathcal{S}$ constructed from the periodic streaming matrix given by Eq. (23). Next, we compute $\kappa_{A_{\text{vor}}}$ for A_{vor} describing the forced Taylor-Green vortex, i.e., again with periodic streaming, but this time with driving captured by Eq. (69). Given that the Carleman driving matrix $\mathcal{D}$ is non-trivial only for $N_C \geq 2$ (as for $N_C = 1$ driving is accounted for only by the Carleman driving vector $\mathcal{F}_0$), we only perform computations for $N_C = 2$ (as for $N_C = 1$ the results are identical to the undriven periodic case). Finally, we compute $\kappa_{A_{\text{lid}}}$ for A_{lid} describing the lid-driven cavity flow, where now both the streaming and driving terms differ from the trivial case. As discussed above, for $N_C = 1$, the driving has no effect and the difference between $\kappa_{A_{\text{lid}}}$ and $\kappa_{A_{\text{per}}}$ stems solely from a modified streaming matrix that now encodes bounce-back conditions at the walls enclosing the simulation region. Interestingly, we observe that in this case, up to numerical precision, $\kappa_{A_{\text{lid}}}$ and $\kappa_{A_{\text{per}}}$ are identical. This can potentially be attributed to the fact that the modification due to walls still preserves the unitary nature of the streaming matrix.

We present the results of our numerical studies for $N_C = 2$ in Fig. 7. To estimate the scaling of κ_A with Re , we perform the best linear fits to:

$$\ln \kappa_A = \chi \ln Re + \ln c, \quad (74)$$

yielding the scaling $O(Re^\chi)$. We extracted the following values of the exponents χ for the three cases investigated:

$$\chi_{\text{per}} = 1.784, \quad \chi_{\text{vor}} = 1.836, \quad \chi_{\text{lid}} = 1.712. \quad (75)$$

Based on the above results, the following three observations can be made. First, given the relatively small number of data points and the fragility of doubly-logarithmic fits, the values of χ for the three considered cases are remarkably

close to each other. This suggests that the algorithm’s asymptotic query complexity for problems with non-trivial geometries and driving should not differ significantly from the simple case with no walls and periodic boundaries. Second, as illustrated by larger values of κ_{lid} as compared to κ_{per} , there still may exist an extra resource cost, but one may expect it to be a constant overhead, not scaling with Re . Third, the classical complexity of solving a two-dimensional CFD problem scales as $O(\text{Re}^{2.25})$, which means that asymptotically our quantum LBM algorithm may outperform the best classical methods by a factor scaling roughly as $O(\text{Re}^{0.45})$. Even including the extra complexity cost $O(\text{Re}^{0.375})$ related to the extraction of the drag force from a quantum-encoded solution (as described in Ref. [38]), should allow one to preserve a non-zero quantum advantage.

V. Conclusion

In this work, we have extended the recent quantum algorithm for the incompressible lattice Boltzmann method to include a class of flows and geometries that are much closer to those encountered in practical CFD, including walls, inlets, outlets, and body forcing. At the algorithmic level, we have shown how these ingredients can be incorporated into the Carleman streaming and driving operators, how they modify the linear system that underpins the quantum linear-solver-based algorithm, and what is the impact on the complexity of solving it. The resulting block-encodings for non-trivial geometries and driving lead to a controlled overhead, scaling as a multiplicative factor of order 4^{N_C} in the query complexity and a linear increase in the Carleman truncation order N_C of ancillary qubits, which preserves the asymptotic scaling, and hence the window for a quantum advantage, identified in earlier work [38].

To assess the practical consequences of these extensions, we implemented a fully matrix-based classical Carleman LBM and benchmarked it against a standard incompressible LBM for three canonical two-dimensional flows: the driven Taylor–Green vortex, the lid-driven cavity flow, and flow past a cylinder. For the forced Taylor–Green vortex, we observed clear evidence of convergence of the Carleman approximation as the truncation order N_C is increased, with $N_C = 3$ yielding a substantial reduction in error, and a relative error that saturates rather than grows in time. For the lid-driven cavity, the Carleman scheme with no-slip walls reproduces the main features of the recirculating flow and the associated shear layers. Moving from $N_C = 1$ to $N_C = 2$ significantly reduces the truncation error, even though the error grows with Reynolds number as the flow structure sharpens near the lid and corners. Finally, for flow past a cylinder at low Reynolds numbers, we demonstrated that moving from $N_C = 1$ to $N_C = 2$ again significantly reduces the truncation error for short times and that the same formalism can handle mixed inlet/outlet conditions and embedded obstacles. Note that a recent work [37] has performed a related Carleman convergence study in a time-continuous lattice Boltzmann setting, i.e., simulating the discrete Boltzmann equation. This was done without the incompressibility assumption, with the Carleman truncation to second order, and for the Kolmogorov and lid-driven cavity flows. Our results are consistent with Ref. [37] (in particular, in both works the relative error as a function of time plateaus), however they also extend and complement this work by including inlet/outlet boundaries and driving.

From the perspective of quantum resources, our numerical study of the condition number of the Carleman linear system indicates that the inclusion of realistic boundaries and driving does not introduce a qualitatively new bottleneck, at least in the parameter ranges investigated. The dominant algorithmic costs remain those already identified for periodic, undriven flows, and the additional overhead from geometry and forcing enters mainly through the block-encoding prefactor and a modest increase in the number of ancillae. This supports the conclusion that, in regimes where the Carleman expansion converges, the end-to-end quantum LBM pipeline can be extended to more realistic CFD setups without closing the potential window for a quantum speedup. However, this window is currently far narrower than what was initially suggested in Ref. [34], which claimed a potential for exponential quantum advantage. Therefore, it strongly motivates more work on pre-conditioning methods, problem-optimized block-encodings, and other algorithmic optimizations. Such advances could have a meaningful impact on the complexity scaling of these Carleman-based algorithms.

Our results also highlight important limitations and open questions. The present study is restricted to two-dimensional, low-Reynolds-number flows and relatively small Carleman orders $N_C \leq 3$, dictated by the classical cost of simulating the embedded linear systems. The observed trends in truncation error with Reynolds number and geometry suggest that more complex flows, particularly three-dimensional turbulence, high-Reynolds-number flows, and strongly time-dependent configurations, will require both improved control of the Carleman convergence radius and more efficient representations of the embedded dynamics. Likewise, our analysis did not include observable extraction beyond the drag force already investigated in Ref. [38], which is an important line to push forward in future work.

Looking forward, several directions appear particularly promising. On the algorithmic side, it will be important to develop alternative linear embedding schemes adapted for particular CFD problems, preconditioning tailored to the LBM

structure, and more economical block-encodings for boundary and forcing operators. On the application side, extending the classical numerical analysis to three dimensions, to higher-Reynolds-number benchmark flows, and eventually to industrially relevant geometries would provide sharper tests of whether practically meaningful quantum advantages can be realized in CFD. In particular, it would be of interest to extend classical simulations of the Carleman LBM implementation to Reynolds numbers at which the transition to time-dependent solutions and vortex shedding emerges, e.g., $Re \approx 83$ for the flow around a cylinder. This would test the ability of the Carleman method to accurately resolve such key physics. More broadly, our results illustrate that end-to-end quantum algorithm designs can be systematically pushed from idealized toy systems towards realistic engineering flows, and that they provide a concrete foundation for future studies at the interface of quantum computing and computational fluid dynamics.

Authors contributions: Authors are listed alphabetically within each affiliation. DJ contributed to the blocking-encoding theory. KK developed extensions to nontrivial flows and geometries, analyzed the corresponding complexity increases, and performed numerical simulations. ML supported the development of extensions to nontrivial flows and geometries. PM assisted in developing extensions to nontrivial flows and geometries, and performed numerical simulations. RA performed numerical studies of truncation errors and operator norms as input to complexity bounds. EM performed numerical comparison between LB and Carleman approximation. SR supported the definition of the end user requirements, and the application use cases. DJ, KK, and PM wrote the paper.

Acknowledgments: We want to thank Thomas Astoul, Thomas Bendokat, Giovanni Giustini, and Alessandro De Rosis for helpful discussions concerning fluid dynamics and the lattice Boltzmann method. We are also grateful to Alice Barthe, Sam Morley-Short, and Trevor Vincent for their assistance with numerical simulations.

Appendix

LBE discrete velocity models

D1Q3	$m = 1:$	$\mathbf{e}_m^* = 0,$	$w_m = 2/3,$
	$m \in \{2, 3\}:$	$\mathbf{e}_m^* \in \{\pm 1\},$	$w_m = 1/6.$
D2Q9	$m = 1:$	$\mathbf{e}_m^* = (0, 0),$	$w_m = 4/9,$
	$m \in \{2, \dots, 5\}:$	$\mathbf{e}_m^* \in \{(\pm 1, 0), (0, \pm 1)\},$	$w_m = 1/9,$
	$m \in \{6, \dots, 9\}:$	$\mathbf{e}_m^* \in \{(\pm 1, \pm 1)\},$	$w_m = 1/36.$
D3Q27	$m = 1:$	$\mathbf{e}_m^* = (0, 0, 0),$	$w_m = 8/27,$
	$m \in \{2, \dots, 7\}:$	$\mathbf{e}_m^* \in \{(\pm 1, 0, 0), (0, \pm 1, 0), (0, 0, \pm 1)\},$	$w_m = 2/27,$
	$m \in \{8, \dots, 19\}:$	$\mathbf{e}_m^* \in \{(\pm 1, \pm 1, 0), (\pm 1, 0, \pm 1), (0, \pm 1, \pm 1)\},$	$w_m = 1/54,$
	$m \in \{20, \dots, 27\}:$	$\mathbf{e}_m^* \in \{(\pm 1, \pm 1, \pm 1)\},$	$w_m = 1/216.$

Table 2 Discrete velocities in lattice units for standard LBE models [41].

Streaming and collision matrices

The streaming matrix S can be explicitly written as:

$$S = \sum_{\mathbf{r}^*} \sum_{m=1}^Q |\mathbf{r}^* + \mathbf{e}_m^* \rangle \langle \mathbf{r}^*| \otimes |m\rangle \langle m|, \quad (76)$$

whereas the linear and quadratic collision matrices, F_1 and F_2 , have the following form:

$$F_1 = \sum_{\mathbf{r}^*} |\mathbf{r}^* \rangle \langle \mathbf{r}^*| \otimes \sum_{m, m_1=1}^Q \frac{1}{\tau^* + \frac{1}{2}} (w_m + 3w_m E_{m, m_1} - \delta_{m, m_1}) |m\rangle \langle m_1|, \quad (77a)$$

$$F_2 = \sum_{\mathbf{r}^*} |\mathbf{r}^* \rangle \langle \mathbf{r}^*, \mathbf{r}^*| \otimes \sum_{m, m_1, m_2=1}^Q \frac{w_m}{\tau^* + \frac{1}{2}} \left(\frac{9}{2} E_{m, m_1} E_{m, m_2} - \frac{3}{2} E_{m_1, m_2} \right) |m \rangle \langle m_1, m_2|, \quad (77b)$$

where $E_{m, n} := \mathbf{e}_m^* \cdot \mathbf{e}_n^*$ and δ is the Kronecker delta.

Carleman streaming and collision matrices

The Carleman streaming matrix S for truncation order N_C is given by the following block-diagonal matrix:

$$S^\infty = \begin{pmatrix} S & 0 & \dots & 0 \\ 0 & S^{\otimes 2} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & S^{\otimes N_C} \end{pmatrix}. \quad (78)$$

The Carleman collision matrix C for truncation order N_C has the following form:

$$C = \begin{pmatrix} C_1^1 & C_2^1 & 0 & 0 & \dots & 0 \\ 0 & C_2^2 & C_3^2 & C_4^2 & \dots & 0 \\ 0 & 0 & C_3^3 & C_4^3 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \dots & C_{N_C}^{N_C} \end{pmatrix}, \quad (79)$$

where in each row k there are $\min\{k + 1, N_C - k + 1\}$ non-zero blocks given by $d^k \times d^l$ matrices:

$$C_l^k := [(I + F_1)^{\otimes 2k-l} \otimes F_2^{\otimes l-k} + \text{perms.}], \quad (80)$$

with + perms. denoting a sum over distinct permutations over k subsystems, with $2k - l$ subsystems of type 1, and $l - k$ subsystems of type 2.

Numerical validation

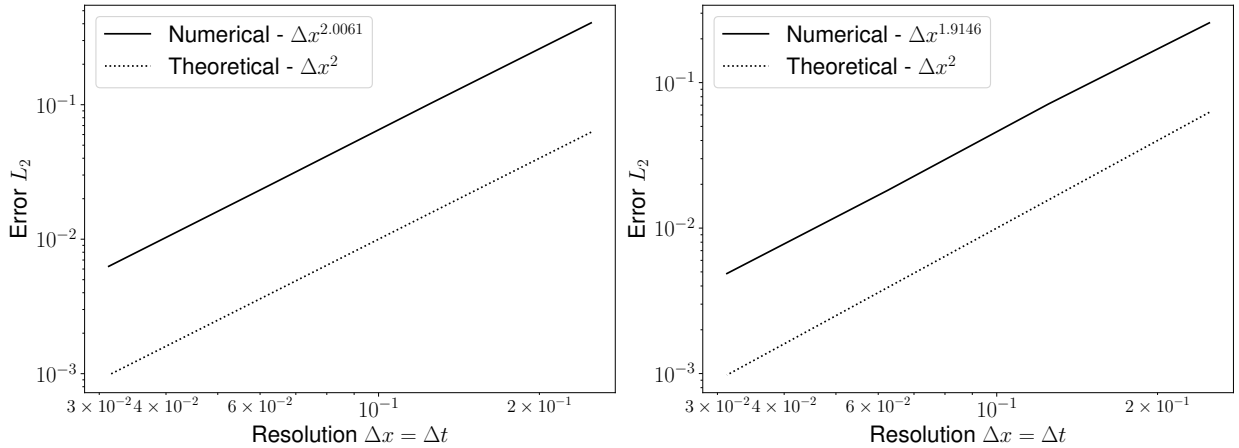


Fig. 8 Validation that our code for $\text{Re} = 10, u_0^* = 0.5$ correctly reproduces (left) the time evolution of the analytical solution for the decaying Taylor-Green vortex and, (right) the analytical solution for the forced Taylor-Green vortex with second-order accuracy in both space and time. Although the forcing used is first-order accurate, the dynamics are steady and we almost recover second order temporal accuracy.

References

- [1] Spalart, P. R., and Venkatakrishnan, V., “On the role and challenges of CFD in the aerospace industry,” *Aeronaut. J.*, Vol. 120, No. 1223, 2016, p. 209–232. <https://doi.org/10.1017/aer.2015.10>.
- [2] Patel, H., Gerhold, T., and Ashton, N., “Assessing the HPC Performance of CODA for the NASA Common Research Model,” *AIAA Aviation Forum and Ascend 2024*, 2024, p. 3792. <https://doi.org/10.2514/6.2024-3792>.
- [3] Paul F. Fischer, J. W. L., and Kerkemeier, S. G., “nek5000 Web page,” , 2008. <http://nek5000.mcs.anl.gov>.
- [4] Bauer, M., Eibl, S., Godenschwager, C., Kohl, N., Kuron, M., Rettinger, C., Schornbaum, F., Schwarzmeier, C., Thönnies, D., Köstler, H., et al., “waLBerla: A block-structured high-performance framework for multiphysics simulations,” *Comput. Math. Appl.*, Vol. 81, 2021, pp. 478–501. <https://doi.org/j.camwa.2020.01.007>.
- [5] Witherden, F. D., and Jameson, A., “Future directions in computational fluid dynamics,” *23rd AIAA Computational Fluid Dynamics Conference*, 2017, p. 3791. <https://doi.org/10.2514/6.2017-3791>.
- [6] Peddinti, R. D., Pisoni, S., Marini, A., Lott, P., Argentieri, H., Tiunov, E., and Aolita, L., “Quantum-inspired framework for computational fluid dynamics,” *Commun. Phys.*, Vol. 7, No. 1, 2024, p. 135. <https://doi.org/10.1038/s42005-024-01623-8>.
- [7] Page, J., “Super-resolution of turbulence with dynamics in the loss,” *J. Fluid Mech.*, Vol. 1002, 2025, p. R3. <https://doi.org/https://doi.org/10.1017/jfm.2024.1202>.
- [8] Tennie, F., Laizet, S., Lloyd, S., and Magri, L., “Quantum computing for nonlinear differential equations and turbulence,” *Nat. Rev. Phys.*, 2025, pp. 1–11. <https://doi.org/10.1038/s42254-024-00799-w>.
- [9] Feynman, R. P., “Simulating physics with computers,” *Feynman and computation*, cRc Press, 2018, pp. 133–153.
- [10] Lloyd, S., “Universal quantum simulators,” *Science*, Vol. 273, No. 5278, 1996, pp. 1073–1078. <https://doi.org/10.1126/science.273.5278.1073>.
- [11] Berry, D. W., Ahokas, G., Cleve, R., and Sanders, B. C., “Efficient quantum algorithms for simulating sparse Hamiltonians,” *Commun. Math. Phys.*, Vol. 270, No. 2, 2007, pp. 359–371. <https://doi.org/10.1007/s00220-006-0150-x>.
- [12] Aspuru-Guzik, A., Dutoi, A. D., Love, P. J., and Head-Gordon, M., “Simulated quantum computation of molecular energies,” *Science*, Vol. 309, No. 5741, 2005, pp. 1704–1707. <https://doi.org/10.1126/science.1113479>.
- [13] McArdle, S., Endo, S., Aspuru-Guzik, A., Benjamin, S. C., and Yuan, X., “Quantum computational chemistry,” *Rev. Mod. Phys.*, Vol. 92, No. 1, 2020, p. 015003. <https://doi.org/10.1103/RevModPhys.92.015003>.
- [14] Harrow, A. W., Hassidim, A., and Lloyd, S., “Quantum Algorithm for Linear Systems of Equations,” *Phys. Rev. Lett.*, Vol. 103, No. 15, 2009, p. 150502. <https://doi.org/10.1103/physrevlett.103.150502>.
- [15] Costa, P. C., An, D., Sanders, Y. R., Su, Y., Babbush, R., and Berry, D. W., “Optimal scaling quantum linear-systems solver via discrete adiabatic theorem,” *PRX Quantum*, Vol. 3, No. 4, 2022, p. 040303. <https://doi.org/10.1103/PRXQuantum.3.040303>.
- [16] Jennings, D., Lostaglio, M., Pallister, S., Sornborger, A. T., and Subaşı, Y., “Efficient quantum linear solver algorithm with detailed running costs,” , 2023. <https://arxiv.org/abs/2305.11352>.
- [17] Danz, S., Berta, M., Schröder, S., Kienast, P., Wilhelm, F. K., and Ciani, A., “Calculating response functions of coupled oscillators using quantum phase estimation,” *Phys. Rev. Res.*, Vol. 7, No. 2, 2025, p. 023264. <https://doi.org/10.1103/5c6d-r1fb>.
- [18] Kiani, B. T., De Palma, G., Englund, D., Kaminsky, W., Marvian, M., and Lloyd, S., “Quantum advantage for differential equation analysis,” *Phys. Rev. A*, Vol. 105, No. 2, 2022, p. 022415. <https://doi.org/10.1103/PhysRevA.105.022415>.
- [19] Rendon, G., “Exponentially Improved Constant in Quantum Solution Extraction,” , 2025. <https://arxiv.org/abs/2508.08375>.
- [20] Berry, D. W., “High-order quantum algorithm for solving linear differential equations,” *J. Phys. A*, Vol. 47, No. 10, 2014, p. 105301. <https://doi.org/10.1088/1751-8113/47/10/105301>.
- [21] Montanaro, A., and Pallister, S., “Quantum algorithms and the finite element method,” *Phys. Rev. A*, Vol. 93, No. 3, 2016, p. 032324. <https://doi.org/10.1103/PhysRevA.93.032324>.
- [22] Berry, D. W., Childs, A. M., Ostrander, A., and Wang, G., “Quantum algorithm for linear differential equations with exponentially improved dependence on precision,” *Commun. Math. Phys.*, Vol. 356, 2017, pp. 1057–1081. <https://doi.org/10.1007/s00220-017-3002-y>.

- [23] Jennings, D., Lostaglio, M., Lowrie, R. B., Pallister, S., and Sornborger, A. T., “The cost of solving linear differential equations on a quantum computer: fast-forwarding to explicit resource counts,” *Quantum*, Vol. 8, 2024, p. 1553. <https://doi.org/10.22331/q-2024-12-10-1553>.
- [24] An, D., Childs, A. M., and Lin, L., “Quantum algorithm for linear non-unitary dynamics with near-optimal dependence on all parameters,” , 2023. <https://arxiv.org/abs/2312.03916>.
- [25] Bagherimehrab, M., Nakaji, K., Wiebe, N., and Aspuru-Guzik, A., “Fast quantum algorithm for differential equations,” , 2023. <https://arxiv.org/abs/2306.11802>.
- [26] Liu, J.-P., and Lin, L., “Dense outputs from quantum simulations,” , 2023. <https://arxiv.org/abs/2307.14441>.
- [27] Carleman, T., “Application de la théorie des équations intégrales linéaires aux systèmes d’équations différentielles non linéaires,” *Acta Math.*, Vol. 59, 1932, pp. 63–87. <https://doi.org/10.1007/BF02546499>.
- [28] Liu, J.-P., Kolden, H. O., Krovi, H. K., Loureiro, N. F., Trivisa, K., and Childs, A. M., “Efficient quantum algorithm for dissipative nonlinear differential equations,” *PNAS*, Vol. 118, No. 35, 2021, p. e2026805118. <https://doi.org/10.1073/pnas.2026805118>.
- [29] Liu, J.-P., An, D., Fang, D., Wang, J., Low, G. H., and Jordan, S., “Efficient quantum algorithm for nonlinear reaction–diffusion equations and energy estimation,” *Commun. Math. Phys.*, Vol. 404, No. 2, 2023, pp. 963–1020. <https://doi.org/10.1007/s00220-023-04857-9>.
- [30] Krovi, H., “Improved quantum algorithms for linear and nonlinear differential equations,” *Quantum*, Vol. 7, 2023, p. 913. <https://doi.org/10.22331/q-2023-02-02-913>.
- [31] Costa, P. C., Schleich, P., Morales, M. E., and Berry, D. W., “Further improving quantum algorithms for nonlinear differential equations via higher-order methods and rescaling,” *npj Quantum Inf.*, Vol. 11, No. 1, 2025, p. 141. <https://doi.org/10.1038/s41534-025-01084-z>.
- [32] Wu, H.-C., Wang, J., and Li, X., “Quantum algorithms for nonlinear dynamics: Revisiting Carleman linearization with no dissipative conditions,” *SIAM J. Sci. Comput.*, Vol. 47, No. 2, 2025, pp. A943–A970. <https://doi.org/10.1137/24M1665799>.
- [33] Jennings, D., Korzekwa, K., Lostaglio, M., Sornborger, A. T., Subasi, Y., and Wang, G., “Quantum algorithms for general nonlinear dynamics based on the Carleman embedding,” , 2025. <https://arxiv.org/abs/2509.07155>.
- [34] Li, X., Yin, X., Wiebe, N., Chun, J., Schenter, G. K., Cheung, M. S., and Mülmenstädt, J., “Potential quantum advantage for simulation of fluid dynamics,” *Phys. Rev. Res.*, Vol. 7, No. 1, 2025, p. 013036. <https://doi.org/10.1103/PhysRevResearch.7.013036>.
- [35] Penuel, J., Katarbwa, A., Johnson, P. D., Farquhar, C., Cao, Y., and Garrett, M. C., “Feasibility of accelerating incompressible computational fluid dynamics simulations with fault-tolerant quantum computers,” , 2024. <https://arxiv.org/abs/2406.06323>.
- [36] Sanavio, C., and Succi, S., “Lattice Boltzmann–Carleman quantum algorithm and circuit for fluid flows at moderate Reynolds number,” *AVS Quantum Science*, Vol. 6, No. 2, 2024. <https://doi.org/10.1116/5.0195549>.
- [37] Turro, F., Lignarolo, A., and Dragoni, D., “Practical Application of the Quantum Carleman Lattice Boltzmann Method in Industrial CFD Simulations,” , 2025. <https://arxiv.org/abs/2504.13033>.
- [38] Jennings, D., Korzekwa, K., Lostaglio, M., Ashworth, R., Marsili, E., and Rolston, S., “An end-to-end quantum algorithm for nonlinear fluid dynamics with bounded quantum advantage,” , 2025. <https://arxiv.org/abs/2512.03758>.
- [39] He, X., and Luo, L.-S., “Lattice Boltzmann model for the incompressible Navier–Stokes equation,” *J. Stat. Phys.*, Vol. 88, No. 3, 1997, pp. 927–944. <https://doi.org/10.1023/B:JOSS.0000015179.12689.e4>.
- [40] Guo, Z., Shi, B., and Wang, N., “Lattice BGK model for incompressible Navier–Stokes equation,” *J. Comput. Phys.*, Vol. 165, No. 1, 2000, pp. 288–306. <https://doi.org/10.1006/jcph.2000.6616>.
- [41] Krüger, T., Kusumaatmaja, H., Kuzmin, A., Shardt, O., Silva, G., and Viggen, E. M., *The Lattice Boltzmann Method: Principles and Practice*, Springer, 2016.
- [42] Bhatnagar, P. L., Gross, E. P., and Krook, M., “A model for collision processes in gases. I. Small amplitude processes in charged and neutral one-component systems,” *Phys. Rev.*, Vol. 94, No. 3, 1954, p. 511. <https://doi.org/10.1103/PhysRev.94.511>.
- [43] Lin, L., “Lecture notes on quantum algorithms for scientific computation,” , 2022. <https://arxiv.org/abs/2201.08309>.

- [44] Junk, M., and Yang, Z., “Outflow boundary conditions for the lattice Boltzmann method,” *Prog. Comput. Fluid Dyn.*, Vol. 8, No. 1-4, 2008, pp. 38–48. <https://doi.org/10.1504/PCFD.2008.018077>.
- [45] Guo, Z., and Shu, C., *Lattice Boltzmann method and its application in engineering*, Vol. 3, World Scientific, 2013.
- [46] Dalzell, A. M., “A shortcut to an optimal quantum linear system solver,” , 2024. <https://arxiv.org/abs/2406.12086>.
- [47] Silva, G., and Semiao, V., “First-and second-order forcing expansions in a lattice Boltzmann method reproducing isothermal hydrodynamics in artificial compressibility form,” *J. Fluid Mech.*, Vol. 698, 2012, pp. 282–303. <https://doi.org/10.1017/jfm.2012.83>.
- [48] Pope, S. B., *Turbulent flows*, Cambridge University Press, 2000.
- [49] Novikau, I., and Joseph, I., “Globalizing the Carleman linear embedding method for nonlinear dynamics,” , 2025. <https://arxiv.org/abs/2510.15715>.